HRFlux: An AI-Powered Multi-Agent HR Assistant.

Project Team

Hadia Mazhar 22I-0487
Shayane Zainab 22I-1049
Fatima Obaid 22I-0475

Session 2022-2026

Supervised by

Dr. Adil Majeed

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.0.1 Product Identification

HRFlux is an AI-based HR automation tool created to make everyday HR work easier and more efficient. Instead of HR teams spending time on small, repetitive tasks, HRFlux takes care of them automatically. It can handle things like answering employee questions, preparing official documents (for example, NOCs or experience letters), sending reminders, and managing follow-ups or escalations.

The system runs on a multi-agent architecture, which basically means it has different AI agents, each responsible for a specific HR function—like handling leaves, giving policy-related answers, or generating documents. This setup helps make sure everything is done accurately, consistently, and on time.

HRFlux also comes with two main interfaces: an employee chatbot, which allows staff to interact with the system directly in real time, and an HR admin dashboard, which HR teams can use for approvals, monitoring, and checking analytics.

1.0.2 Intended Audience

This document is written for different people who are directly or indirectly involved in creating, testing, or using HRFlux:

1. Developers: They'll use this document to understand how the system works behind the scenes—its technical setup, how the architecture is built, and how different parts like the backend, AI modules, and frontend connect with each other.

2. Project Managers: For project managers, this document helps them keep track of progress, manage timelines, assign resources, and make sure the whole project is going according to plan.

3. Testers: Testers will rely on this document to know what features need to be checked. They'll design and run test cases to confirm everything works properly and meets both functional and non-functional requirements.

4. Users (HR Staff and Employees): Once the system is live, HR officers, managers, and regular employees will be the main users. This document gives them a clear picture of what HRFlux does, how it can make their daily work smoother, and what kind of interactions they can expect.

5. Documentation Writers: Writers will use this as a guide while preparing user manuals, installation instructions, and training materials so others can easily learn how to use and maintain HRFlux.

6. Stakeholders (Legal IT Teams): Legal teams may review this document to ensure everything complies with company policies and data protection laws. Meanwhile, IT or DevOps teams will focus more on how to deploy the system, handle API integrations, and keep the data secure.

In short, this document is meant to serve as a one-stop reference for everyone involved in the HRFlux project—whether they're technical or not. It helps make sure that every team member clearly understands what the product is, what it's supposed to do, and how all parts fit together. [7].

1.1 Existing Solutions

Several existing HR automation systems aim to streamline employee support, document handling, and HR operations. However, each system has certain strengths and weaknesses that reveal clear gaps in the industry. This section reviews notable existing solutions and highlights the limitations that the proposed system, **HRFlux**, aims to address.

Deel AI Personas provides strong conversational capabilities with specialized digital agents and basic multi-agent collaboration. It enables automated employee interactions but lacks document auto-generation and proactive intelligence, reducing its overall automation capability.

Workday Illuminate offers specialized AI agents integrated within the Workday ecosystem. While powerful in structured enterprise environments, it has limited cross-platform compatibility and weak multi-agent collaboration features, which restrict flexibility for diverse HR workflows.

Leena AI supports cross-platform automation and provides document generation tools, which make it useful for employee support. However, it lacks true multi-agent collaboration and proactive intelligence, which limits its ability to independently resolve complex HR tasks.

HRFlux brings together all the missing pieces by introducing specialized agents, multi-agent collaboration, cross-platform integration, document auto-generation, and proactive intelligence, enabling a more comprehensive and autonomous HR automation solution.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Deel AI Personas	Provides specialized conversational agents and supports multi-agent collaboration for HR-related dialogues.	Lacks document auto-generation and proactive intelligence, limiting automation depth.
Workday Illuminate	Enterprise-grade AI agents integrated into the Workday platform, offering structured HR support and analytics.	Limited cross-platform support and weak multi-agent collaboration, reducing flexibility across various HR environments.
Leena AI	Cross-platform HR automation tool offering chat-based support and document generation.	Does not support true multi-agent collaboration and lacks proactive intelligence for complex autonomous task execution.
HRFlux	A unified HR automation system with specialized agents, multi-agent collaboration, cross-platform support, document auto-generation, and proactive intelligence.	System is under development; real-world performance and scalability are yet to be validated.

1.2 Problem Statement

In most organizations, HR teams spend a huge chunk of their time doing the same repetitive tasks over and over again. Things like answering questions about leaves, preparing NOCs, or explaining company policies might seem small, but they pile up fast. Every week, HR staff end up spending hours on these routine activities, which means they have less time for the more important stuff—like improving policies, helping employees grow, or planning long-term HR strategies.

For employees, getting simple answers can also be a headache. Information is often buried inside long documents or on old, messy intranet pages. So instead of quick solutions, they end up sending multiple emails or asking the same questions again and again. This

not only wastes time but also causes frustration and confusion, especially when different people give slightly different answers.

To make things worse, many existing HR chatbots or FAQ tools don't really help. They usually give generic, copy-paste-style replies that don't match what employees actually want to know.

Another big issue is with HR documents. Most of them—like experience letters, approval memos, or confirmation letters—are still prepared manually. This process is slow, and mistakes happen easily. Without automation, it's also hard to track reminders or escalations properly, which means approvals get delayed and SLAs (service-level agreements) are often missed. These delays and inconsistencies can even lead to compliance or audit problems later on. Small HR teams struggle the most because even a small rise in employee queries can completely overwhelm their workload.

That's where HRFlux comes in. The main goal behind developing this system is to solve all these everyday HR challenges. HRFlux uses AI to automate routine tasks, manage documents, and give fast, accurate answers based on company policies. It also keeps track of reminders, approvals, and escalations on its own, making sure everything happens on time. By reducing the manual work and improving how HR communicates, HRFlux helps HR professionals focus on things that actually matter—while employees get quicker, clearer, and more reliable support.

1.3 Scope

This project focuses on creating HRFlux, an AI-based system designed to make HR work faster, smarter, and easier for everyone. The main idea is to automate the everyday HR tasks that usually take up a lot of time—like answering employee questions, managing leave requests, generating documents, and sending reminders or escalations.

HRFlux is built around a multi-agent setup, which basically means there are different AI "bots" that each handle a specific part of HR work. For example, LeaveBot looks after leave management, PolicyBot helps with company rules and policies, and DocuBot takes care of document creation. This approach helps make the system more accurate and reliable since each agent focuses on what it does best.

The system will also have a semantic knowledge base—a smart search space where employees can find information straight from official HR documents. This ensures that whatever answers the system gives are always based on real company policies, not random or outdated info.

Employees will be able to talk to HRFlux through a chatbot, just like chatting with a person. They can ask questions, apply for leave, or request documents in natural lan-

guage. On the HR team's side, there will be an admin dashboard where staff can review or approve responses, update policies, and check system performance.

Another key feature is document automation. HRFlux will generate standard letters like NOCs, experience letters, and approval notes using pre-made templates, cutting down on repetitive typing and formatting. It will also handle reminders and escalations automatically so that important tasks or deadlines aren't forgotten.

To make everything fit smoothly into an organization, HRFlux will be able to connect with existing HR systems (like HRIS or payroll platforms) using APIs. This allows it to read and update employee or leave data without manual effort. On top of that, it will include essential security measures like role-based access control, audit trails, and compliance with data privacy rules.

The scope of this project covers building the AI agents, backend services, chatbot interface, admin dashboard, and database. However, it won't include complete integration with multiple enterprise systems—only one example connection with an HRIS will be shown.

By the end, the goal is to have a working prototype that clearly demonstrates how HRFlux can make HR operations faster, more consistent, and more efficient. This prototype will serve as a strong starting point for future upgrades and deployment in real organizations.

1.4 Modules

The proposed system, **HRFlux**, is designed in a modular way to ensure scalability, clarity, and efficient development. Each module focuses on a specific set of functionalities, allowing independent development and testing while collectively achieving the project goals. The modules are grouped according to system type: employee interface, HR admin interface, and backend AI services.

1.4.1 Module 1: Employee Interface (Client Web/Mobile App)

This module provides employees with a user-friendly interface to interact with HRFlux, submit requests, and receive instant responses to HR queries.

1. **Employee Chatbot:** Handles routine queries like leave balances, policy clarifications, and benefits using contextual session memory.

2. **Document Requests:** Allows employees to request NOCs, experience letters, or approval memos, which are automatically generated using templates.

3. **Notifications & Reminders:** Sends proactive alerts about pending approvals, deadlines, or policy updates.

1.4.2 Module 2: HR Admin Interface (Admin Web App)

This module allows HR staff to monitor, manage, and approve employee requests while maintaining compliance and workflow efficiency.

1. **Admin Dashboard:** Review, approve, or modify employee requests and generated documents; track metrics like resolution times and escalations.

2. **Query Logs & Monitoring:** Maintain records of all employee interactions for auditing and performance tracking.

3. **Escalation Management:** Route unresolved or sensitive queries to the right HR officer with full conversation history for smooth handover.

1.4.3 Module 3: Backend and AI Services

This module handles data storage, AI-driven processing, and integration with existing HR systems to automate workflows.

1. **Data Collection & Database Management:** Store employee profiles, leave balances, payroll information, and policy documents in a structured database.

2. **Knowledge Base & Search:** Embed HR documents into a vector database to support semantic search and retrieval of accurate answers.

3. **HR Agent Layer:** Includes specialized agents such as LeaveBot, PolicyBot, DocuBot, and EscalationBot for handling specific HR tasks.

4. **Workflow & System Integration:** Automate leave approvals, document generation, notifications, and integrate with HRIS/payroll systems via APIs.

1.5 User Classes and Characteristics

User Class	Description
Employee	Employees are the primary users who interact with HRFlux through the chatbot interface. They submit leave requests, ask policy-related questions, and request documents such as NOCs or experience letters. Employees vary in tech-savviness but generally use the system via web or mobile devices.
HR Manager/Officer	Responsible for reviewing and approving employee requests. They use the admin dashboard to monitor document workflows, track performance, and ensure compliance. They need a simple interface and access to detailed logs for accountability.
IT/DevOps Staff	Manage integration, deployment, and maintenance of HRFlux. They ensure API connectivity, backend performance, and system security. They require access controls, documentation, and tools for troubleshooting.
Legal/Compliance Team	Review policies, ensure privacy and legal compliance, and audit document histories. They use the system occasionally but require clear logs and secure access.

1.6 Work Division

The development of **HRFlux** is divided among the team members to ensure focused implementation of core features. The responsibilities are distributed as follows:

1.6.1 Hadia Mazhar – Core Agents & Knowledge

- **Develop Specialized AI Agents:** Implementing LeaveBot and PolicyBot for handling specific domain queries.

- **Implement Agent Collaboration:** Designing multi-step workflows where agents interact to solve complex requests.

- **Build Knowledge Base Search:** Developing the system for uploading and semantically searching policy documents.

1.6.2 Shayane Zainab – Experience & Integration

- **Build Smart Escalation Routing:** Creating logic to route unresolved queries to the appropriate HR officer.
- **Implement Context Retention (Memory):** Ensuring the system remembers conversation history for personalized answers.
- **Develop Unified Portal:** Building the frontend and integration for both the employee chatbot and the admin dashboard.

1.6.3 Fatima Obaid – Automation & Workflow

- **Implement End-to-End Leave Workflow:** Managing the entire lifecycle of leave applications from submission to approval and tracking.
- **Develop Auto-Document Generation:** Automating the creation of official documents like NOCs and leave approval letters.
- **Add Proactive Notifications:** Implementing reminders and alerts regarding status updates and deadlines.

Chapter 2

Project Requirements

2.1 Use-case Diagram/Table:

The Use Case Diagram for HRFlux provides a high-level view of how different users interact with the system. It captures the main functionalities available to each actor and outlines the system's overall behavior in response to user actions. The primary goal is to visualize user interactions, showing what each user type can do within the system.

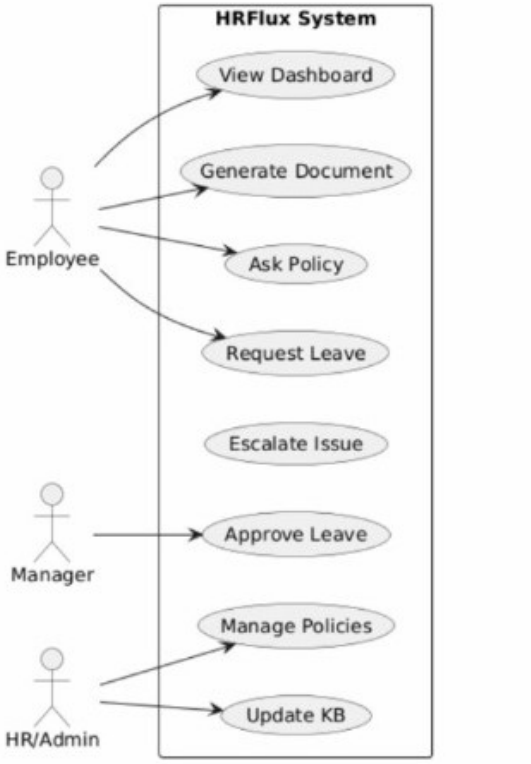


Figure 2.1: Use Case Diagram for HRFlux

Description: The HRFlux system involves three main actors: **Employee**, **HR Manager**,

and **Admin System**. Each actor interacts with the system through distinct use cases representing their roles and responsibilities.

- **Employee:** Employees can interact with the HRFlux chatbot to perform core HR tasks such as submitting leave requests, viewing policies, requesting official documents, or checking the status of their requests. The chatbot acts as the main interface for these interactions.

- **HR Manager:** The HR Manager reviews and approves requests, manages escalations, updates HR policies, and oversees workflow through the admin portal. They are responsible for ensuring timely approvals and policy compliance.

- **Admin System:** The Admin System handles backend operations such as policy ingestion, document template updates, analytics generation, and synchronization with the external HRIS system to maintain accurate records.

- **Use Cases:** Key use cases include “Submit Leave Request,” “Query HR Policy,” “Generate Document,” “Approve Request,” “Handle Escalation,” “Update Knowledge Base,” and “Export Analytics.” These cover the major functionalities that define the HRFlux platform.

The use case diagram effectively demonstrates how HRFlux integrates conversational AI and automation to streamline HR operations, reducing manual workload while maintaining efficient communication between employees and HR personnel.

Since HRFlux is an interactive end-user application, a **use-case approach** is suitable. The following table summarizes the main actors, use cases, and descriptions.

Use Case ID	Actor	Description
UC-01	Employee	Submit leave requests through the chatbot. The system validates, updates HRIS, and confirms the request.
UC-02	Employee	Ask HR policy questions. The system retrieves answers from the knowledge base and provides context-aware responses.
UC-03	Employee	Request documents like NOCs, experience letters, or approval memos. DocuBot generates and delivers the document.
UC-04	Employee	Receive reminders and notifications for pending approvals or deadlines.
UC-05	HR Manager/Officer	Review and approve/reject leave requests or document requests via admin dashboard.
UC-06	HR Manager/Officer	Track employee queries, escalations, and system performance metrics.
UC-07	HR Manager/Officer	Modify or override generated documents to maintain compliance.
UC-08	IT/DevOps	Maintain integrations with HRIS/payroll systems, monitor backend, and ensure system security.
UC-09	Legal/Compliance	Audit workflows, documents, and queries for regulatory and organizational compliance.

2.2 Functional Requirements

This section lists the functional requirements for HRFlux, organized by module. Each requirement describes a specific functionality of the system in natural language.

2.2.1 Module 1: Employee Interface (Client Web/Mobile App)

Following are the functional requirements for the employee interface:

1. FR1: Employees can submit leave requests through a chatbot interface. The system validates the request and updates the HRIS.

2. FR2: Employees can ask questions related to HR policies and receive accurate, context-aware responses from the knowledge base.

3. FR3: Employees can request official documents such as NOCs, experience letters, or approval memos, which are auto-generated using predefined templates.

4. FR4: Employees receive proactive notifications and reminders for pending approvals, deadlines, and policy updates.

2.2.2 Module 2: HR Admin Interface (Admin Web App)

Following are the functional requirements for the HR admin interface:

1. FR1: HR staff can review, approve, or reject leave requests and document requests submitted by employees.
2. FR2: HR staff can monitor queries, escalations, and system performance metrics via the admin dashboard.
3. FR3: HR staff can modify or override generated documents to ensure compliance with company policies.
4. FR4: The system logs all interactions and maintains query histories for auditing and accountability.

2.2.3 Module 3: Backend and AI Services

Following are the functional requirements for the backend and AI modules:

1. FR1: Store and manage employee profiles, leave balances, payroll information, and HR policies in a structured database.
2. FR2: Implement a knowledge base with semantic search for accurate retrieval of policy-based answers.
3. FR3: Include specialized AI agents (LeaveBot, PolicyBot, DocuBot, EscalationBot) to handle specific HR tasks automatically.
4. FR4: Automate workflow processes, notifications, and integration with HRIS/payroll systems via APIs.

2.3 Non-Functional Requirements

This section specifies non-functional requirements for HRFlux. These requirements focus on system qualities such as reliability, usability, performance, and security. Each requirement is specific, measurable, and verifiable.

2.3.1 Reliability

The system must operate continuously with minimal failures and recover gracefully in case of errors. Reliability requirements include:

- REL-1: HRFlux shall have a mean time between failures (MTBF) of at least 500 hours under normal operational load.
- REL-2: In the event of a system failure, ongoing user sessions shall be recoverable without loss of data within 2 minutes.
- REL-3: Automated error detection and logging shall capture all failures and notify the IT/DevOps team within 5 minutes.

2.3.2 Usability

HRFlux should provide an intuitive and efficient user experience for employees and HR staff. Usability requirements include:

- USE-1: Employees shall be able to submit a leave request or document request using the chatbot in under 2 minutes.
- USE-2: New users shall be able to learn basic operations of the system within 15 minutes without external help.
- USE-3: The system shall provide clear guidance, error messages, and recovery options if a user enters invalid information.
- USE-4: The interface shall be accessible on both web and mobile platforms with responsive design.

2.3.3 Performance

HRFlux should respond quickly to user interactions and maintain smooth operation under expected workloads. Performance requirements include:

- PER-1: 95% of chatbot responses shall be delivered within 3 seconds of user input over a 10 Mbps or faster Internet connection.
- PER-2: Document generation (NOC, experience letter) shall complete within 5 seconds for a standard template.
- PER-3: The system shall handle at least 500 concurrent employee sessions without performance degradation.

2.3.4 Security

HRFlux must protect user data and system integrity against unauthorized access. Security requirements include:

- SEC-1: All employee and HR data shall be encrypted at rest and in transit.
- SEC-2: Role-based access control shall ensure that only authorized users can view, modify, or approve HR records.
- SEC-3: All system interactions shall be logged for auditing purposes, including document generation, leave approvals, and policy queries.
- SEC-4: The system shall resist unauthorized attempts to access data or system resources, with alerts triggered on suspicious activity.

Chapter 3

System Overview

HRFlux is an AI-powered HR automation system designed to simplify routine HR tasks and improve the employee experience. The system allows employees to submit leave requests, ask HR policy-related questions, and request official documents, while HR staff can approve requests, monitor workflows, and ensure compliance. HRFlux is designed as a multi-agent system with specialized modules, including a chatbot interface for employees, an admin portal for HR staff, backend services, and AI-driven agents. The system integrates with HRIS and payroll platforms through APIs to automate data updates and maintain consistent records.

3.1 Architectural Design

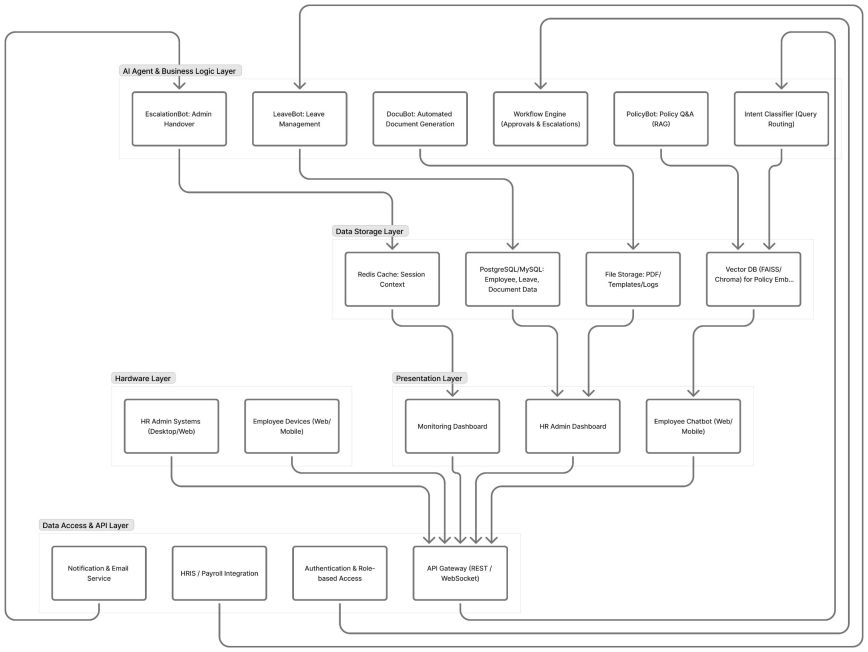


Figure 3.1: HRFlux System Architecture Diagram

The architecture diagram illustrates the layered structure of HRFlux, showing how the presentation, application, data, and integration layers interact to deliver seamless HR automation and communication between employees, HR staff, and AI agents.

The architectural design of HRFlux follows a modular approach, allowing independent development, testing, and scaling of each module. The main modules include:

- **Employee Interface:** A chatbot that interacts with employees to handle leave requests, policy queries, and document requests.
- **HR Admin Interface:** A web portal for HR staff to approve requests, monitor performance, and manage employee data.
- **AI Agent Layer:** Specialized agents (LeaveBot, PolicyBot, DocuBot, EscalationBot) that process queries, generate documents, and escalate unresolved issues.
- **Backend Services:** Data management, workflow engine, knowledge base, and integration with HRIS/payroll systems.

- **Notification Reminder System:** Sends proactive notifications to employees and HR staff regarding pending tasks, deadlines, and SLA compliance.

The modules interact through a simple coordination layer to ensure smooth workflow and maintain session context. This modular structure makes the system robust, scalable, and easy to maintain.

3.1.1 Box-and-Line Diagram (High-Level)

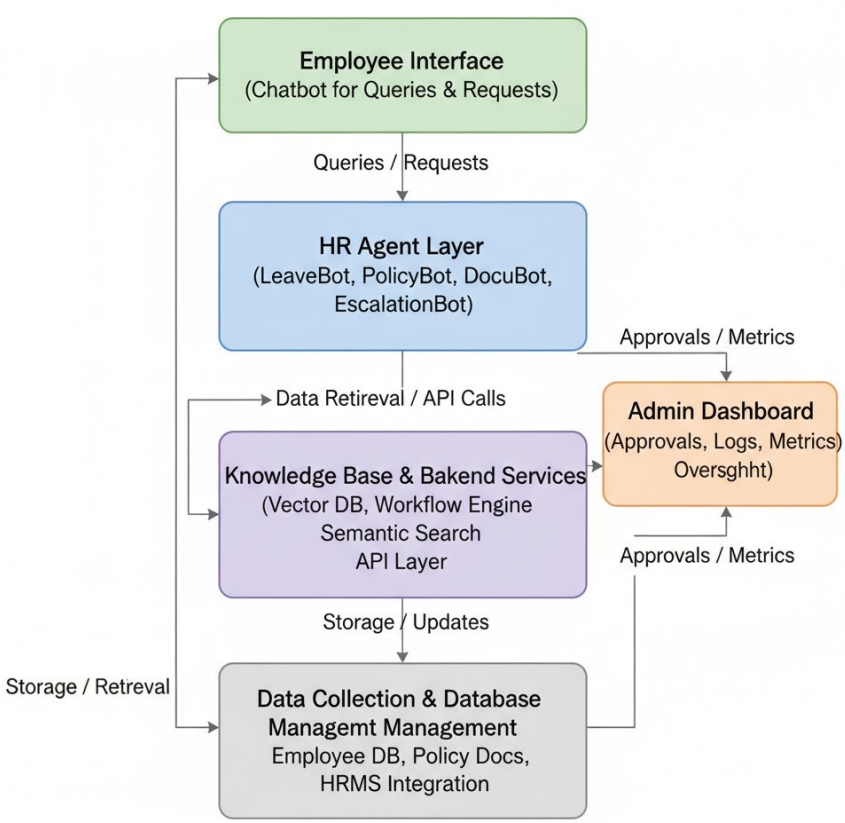


Figure 3.2: Box-and-Line Diagram for HRFlux

This diagram shows how different HRFlux components interact. Employees use the chatbot to send queries that go through the AI Agent Layer. The agents communicate with backend services for data processing and HRIS updates. HR admins monitor activity via the dashboard, while the notification system keeps both users informed with reminders and alerts.

3.1.2 Architecture Style

HRFlux follows a **Layered Multi-tier Architecture**:

- **Presentation Layer:** Employee chatbot and HR admin dashboard.
- **Application Layer:** AI agents that handle requests, generate documents, and escalate cases.
- **Data Layer:** Backend services including database, workflow engine, and knowledge base.
- **Integration Layer:** Interfaces with external systems such as HRIS and payroll platforms via APIs.

This layered architecture allows clear separation of concerns, improves maintainability, and enables future scalability of HRFlux.

3.2 Data Design

The data design of HRFlux defines how the system's information domain is represented and stored in a structured manner to support efficient operations. The system deals with employee records, HR policies, leave requests, document requests, and system logs. Each of these entities is stored in well-defined data structures and databases to ensure data integrity, fast retrieval, and secure management.

3.2.1 Major Data Entities

- **Employee:** Stores details such as employeeID, name, department, email, role, leave balance, and employment history. This information is required to process leave requests, document requests, and access control.
- **LeaveRequest:** Contains requestID, employeeID, leave type, start and end dates, status (pending, approved, rejected), and approval history. This entity is essential for tracking leave workflows and SLA compliance.
- **DocumentRequest:** Stores requestID, employeeID, document type (NOC, experience letter, approval memo), request status, generation timestamp, and approval details.
- **Policy:** Maintains policyID, title, content, effective date, and related attachments. Policies are indexed for semantic search and retrieval by PolicyBot.

- **AgentSession:** Tracks sessionID, employeeID, conversation context, previous queries, and memory state. This ensures personalized multi-step interactions with the chatbot.
- **Audit Logs:** Captures all system activities, including leave approvals, document generation, policy queries, and escalations, along with timestamps and user IDs for compliance and monitoring purposes.

3.2.2 Data Storage and Processing

HRFlux uses a combination of structured and semi-structured data storage:

- **Relational Database:** MySQL or PostgreSQL is used to store structured data, including employee records, leave requests, document requests, and audit logs. These databases support transactional integrity and relational queries.
- **Vector Database:** FAISS or Chroma is used to store embedded representations of policy documents and FAQs for semantic search, allowing agents to retrieve contextually accurate answers quickly.
- **Document Storage:** PDF or Word documents (e.g., policies, NOCs, experience letters) are stored in a secure file system or object storage, with metadata stored in the relational database for quick access.

3.2.3 Data Relationships

The system's entities are interrelated to ensure smooth operation:

- Each **Employee** can have multiple **LeaveRequests** and **DocumentRequests**.
- **Policy** documents are linked to employee queries via semantic search for accurate responses.
- **Audit Logs** maintain relationships with employees, HR staff, and agents for accountability.
- **AgentSession** links conversations to employees to maintain context across multiple interactions.

This data design ensures that HRFlux can efficiently manage HR operations, maintain consistency and compliance, and provide a fast, personalized experience for employees while supporting HR staff in decision-making and workflow management.

3.3 Domain Model

The Domain Model (or Class Diagram) for HRFlux represents the key entities, their attributes, and the relationships that define the system's structure. It follows an object-oriented approach to capture the logical architecture of the multi-agent HR assistant.

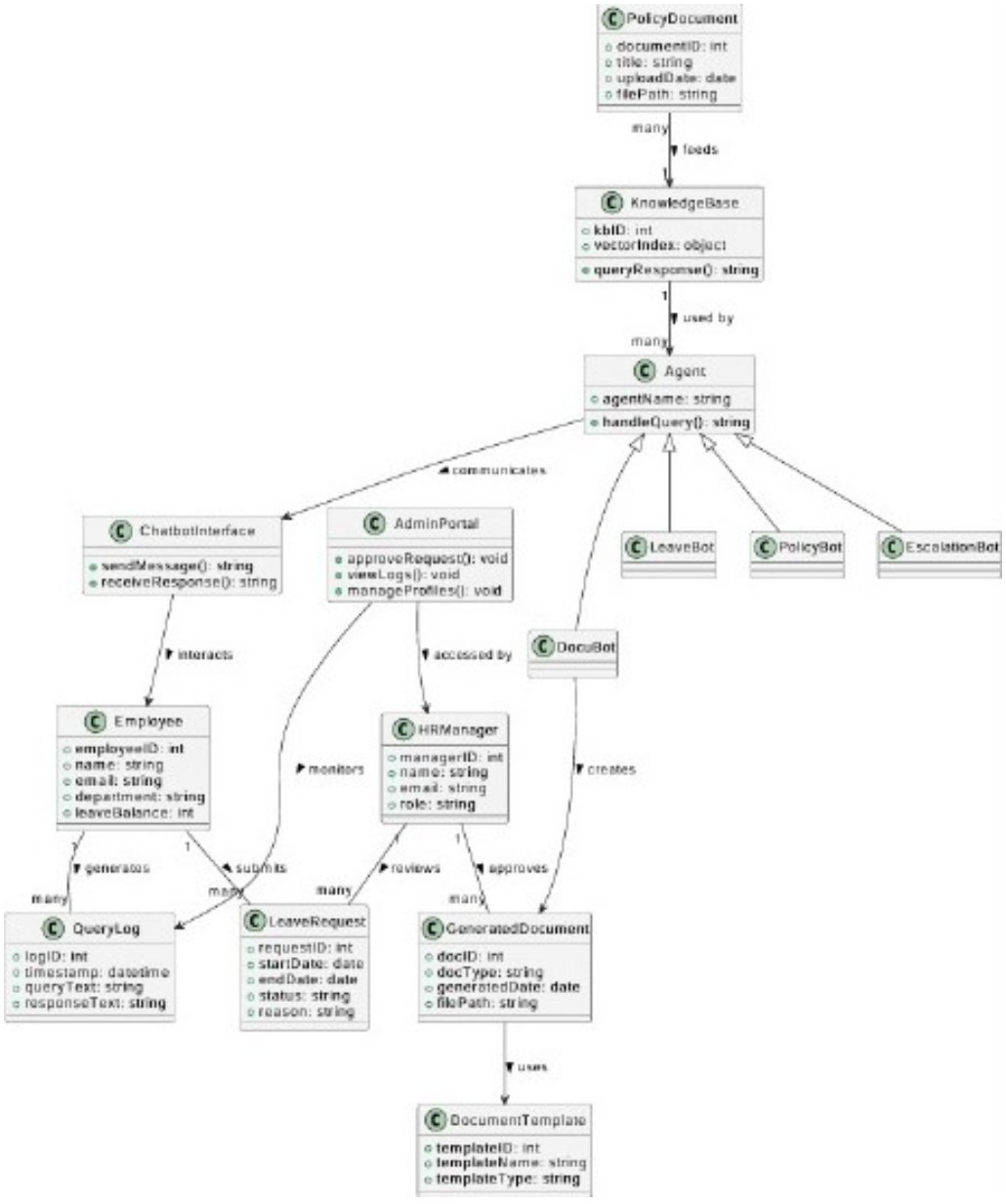


Figure 3.3: Domain Model (Class Diagram) for HRFlux

Description: The domain model captures the main entities of the HRFlux system and their interactions. It defines the structure and associations among the Chatbot Interface, Admin Portal, Agents, Knowledge Base, and document management classes. Below is a detailed explanation of the components:

- **Employee:** Represents an HRFlux end-user who interacts through the chatbot to make requests such as leaves, document generation, or policy inquiries.
  - *Attributes:* employeeID, name, email, department, status.
  - *Methods:* sendMessage(), receiveResponse().
- **ChatbotInterface:** Acts as the primary communication channel between the employee and the system. It collects user input and forwards it to the Agent layer.
  - *Attributes:* None specified.
  - *Methods:* sendMessage(), receiveResponse().
- **Agent (Abstract Class):** A general representation of all intelligent agents in HRFlux. Each specialized agent inherits from this class and implements its own query-handling logic.
  - *Attributes:* agentName, agentID.
  - *Methods:* handleQuery().
- **LeaveBot, PolicyBot, EscalationBot:** Subclasses of the Agent class, each responsible for specific HR tasks:
  - *LeaveBot:* Processes leave requests and interacts with HR Manager for approval.
  - *PolicyBot:* Retrieves policy-related answers via the Knowledge Base.
  - *EscalationBot:* Routes complex queries or complaints to the Admin Portal.
- **DocuBot:** Another specialized agent responsible for generating HR documents using pre-defined templates and employee data. It interacts with the DocumentTemplate and GeneratedDocument classes.
- **KnowledgeBase:** The information retrieval backbone of HRFlux. It uses vector-based embeddings to answer policy queries (RAG flow).
  - *Attributes:* kbID, vectorIndex, queryResponse.
  - *Methods:* queryResponse().
- **PolicyDocument:** Contains structured policy data that feeds into the Knowledge Base.
  - *Attributes:* documentID, title, filePath, lastUpdate.
- **AdminPortal:** Interface for HR managers and admins to review escalations, approve leave requests, and manage HR processes.

  - *Attributes:* adminID.
  - *Methods:* approveRequests(), viewLogs(), manageProfiles().

- **HRManager:** Represents the decision-maker responsible for approving leaves, reviewing generated documents, and handling escalations.
  - *Attributes:* managerID, name, email, role, status.
  - *Methods:* approve(), review().

- **LeaveRequest:** Captures employee leave request details.
  - *Attributes:* requestID, requestType, status, reason, date.
  - *Methods:* submit(), updateStatus().

- **GeneratedDocument:** Represents auto-generated HR documents such as leave letters or experience certificates.
  - *Attributes:* docID, docType, dateCreated, filePath.
  - *Methods:* finalize(), store().

- **DocumentTemplate:** Defines reusable templates used by DocuBot for generating documents.
  - *Attributes:* templateID, templateName, templatePath.

- **QueryLog:** Stores chat session history and user queries for analytics and context restoration.
  - *Attributes:* logID, timestamp, queryText, responseText.

Relationships:

- Each **Employee** interacts with the **ChatbotInterface** to generate **QueryLogs** and initiate **LeaveRequests**.
- The **Agent** hierarchy (LeaveBot, PolicyBot, EscalationBot, DocuBot) handles queries routed from the chatbot.
- The **KnowledgeBase** retrieves data from the **PolicyDocument** for query answering.
- **HRManager** interacts through the **AdminPortal** to approve **LeaveRequests** and **GeneratedDocuments**.
- The **DocuBot** uses **DocumentTemplate** to create **GeneratedDocuments**.

Summary: This domain model integrates chatbot communication, multi-agent logic, HR workflows, and knowledge retrieval into a unified structure. It ensures modularity, easy scalability, and clear separation of responsibilities between user interface, business logic, and data persistence layers.

3.4 Design Models (Procedural Approach)

This section presents the design models for the HRFlux system based on the procedural development approach. The following diagrams describe the flow of control, data communication, and process handling throughout the system.

3.4.1 Activity Diagram

The activity diagram illustrates the overall functional flow of the HRFlux system. It represents how a user query or request moves through different system modules until a response is generated and logged.

1. Employee submits query or request.
2. System receives and temporarily stores input.
3. Intent classification is performed (e.g., leave, policy, document).
4. Query is routed to the respective processing module.
5. If administrative action is required, the request is escalated.
6. System generates, retrieves, or formats the response.
7. Response is sent back to the employee.
8. Interaction is logged for analytics.

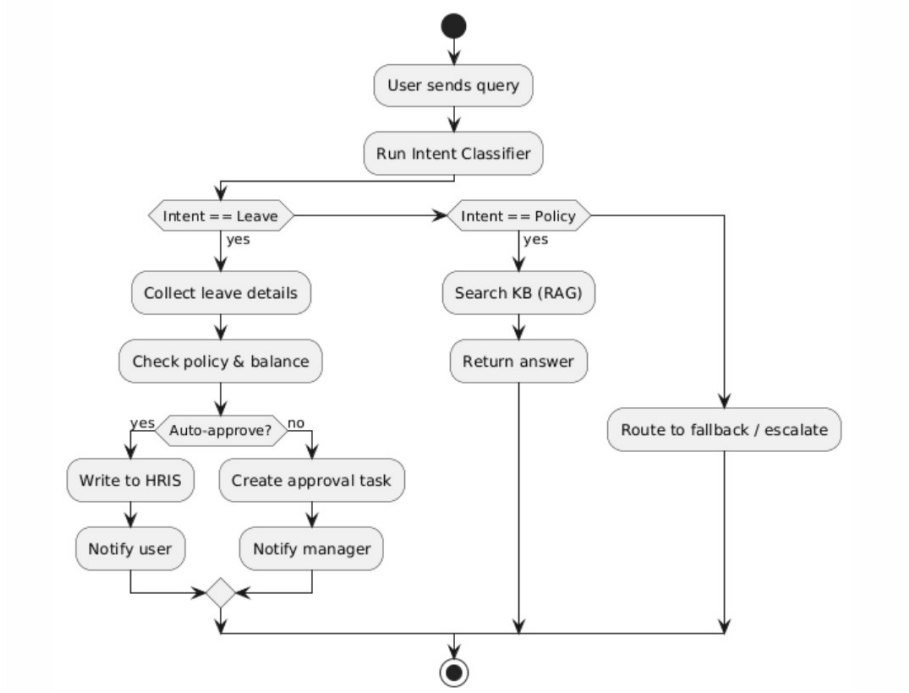


Figure 3.4: Activity Diagram for HRFlux

3.4.2 Sequence Diagram — Leave Request

This diagram represents the flow of a leave request from submission to approval in HRFlux.

1. Employee → InputHandler : Submit leave request
2. InputHandler → IntentClassifier : Classify query type
3. IntentClassifier → LeaveProcessor : Forward leave details
4. LeaveProcessor → DatabaseModule : Validate employee data & leave balance
5. DatabaseModule → LeaveProcessor : Return validation result
6. LeaveProcessor → HRManager : Send approval request
7. HRManager → LeaveProcessor : Approve/Reject leave
8. LeaveProcessor → Notifier : Send final response

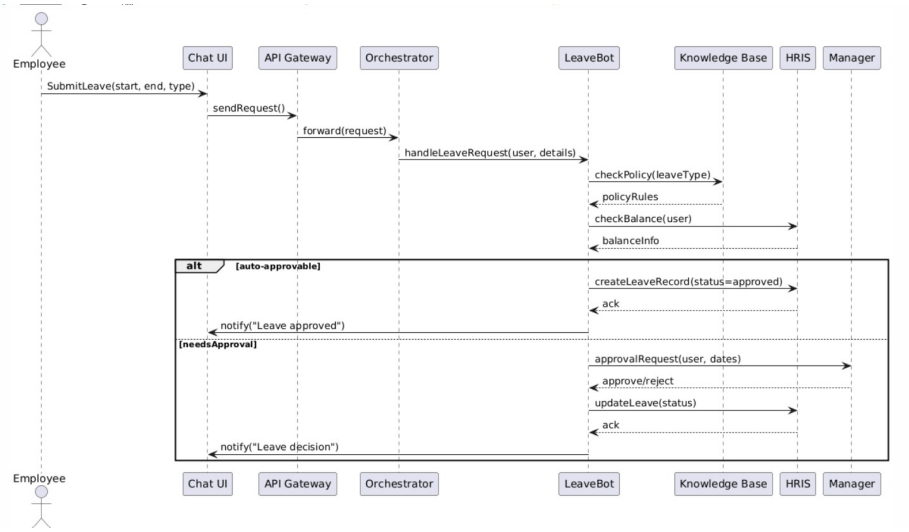


Figure 3.5: Sequence Diagram — Leave Request

3.4.3 Sequence Diagram — Policy Query (RAG Flow)

This sequence shows how HRFlux retrieves policy documents and generates a contextual AI response.

1. Employee → InputHandler : Submit policy question
2. InputHandler → IntentClassifier : Detect policy query
3. IntentClassifier → RAGModule : Trigger retrieval process
4. RAGModule → KnowledgeBase : Retrieve relevant policies
5. KnowledgeBase → RAGModule : Return text passages
6. RAGModule → ResponseGenerator : Generate summarized response
7. ResponseGenerator → Notifier : Send final answer

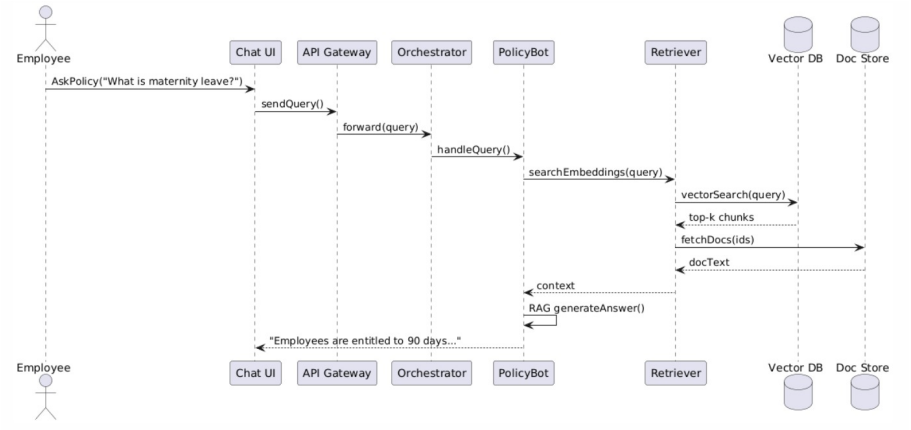


Figure 3.6: Sequence Diagram — Policy Query (RAG Flow)

3.4.4 Sequence Diagram — Document Generation

This process illustrates automated generation of HR documents like NOCs or experience letters.

1. Employee → InputHandler : Request document
2. InputHandler → IntentClassifier : Identify document generation intent
3. IntentClassifier → DocumentGenerator : Load document template
4. DocumentGenerator → DatabaseModule : Fetch employee data
5. DatabaseModule → DocumentGenerator : Return details
6. DocumentGenerator → Formatter : Generate final document
7. Formatter → Notifier : Deliver document to user

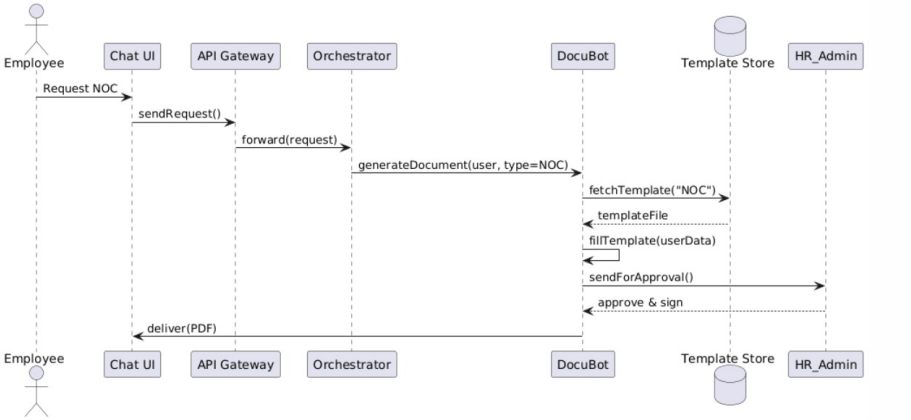


Figure 3.7: Sequence Diagram — Document Generation

3.4.5 Sequence Diagram — Escalation Flow (Bot → Admin)

When the AI system cannot resolve a query, it escalates it to an HR admin.

1. Employee → InputHandler : Send ambiguous query
2. InputHandler → IntentClassifier : Detect low-confidence intent
3. IntentClassifier → EscalationManager : Trigger escalation
4. EscalationManager → AdminDashboard : Forward query
5. AdminDashboard → HRAdmin : Notify assigned admin
6. HRAdmin → AdminDashboard : Provide manual resolution
7. AdminDashboard → ResponseGenerator : Generate reply
8. ResponseGenerator → Employee : Deliver resolved answer

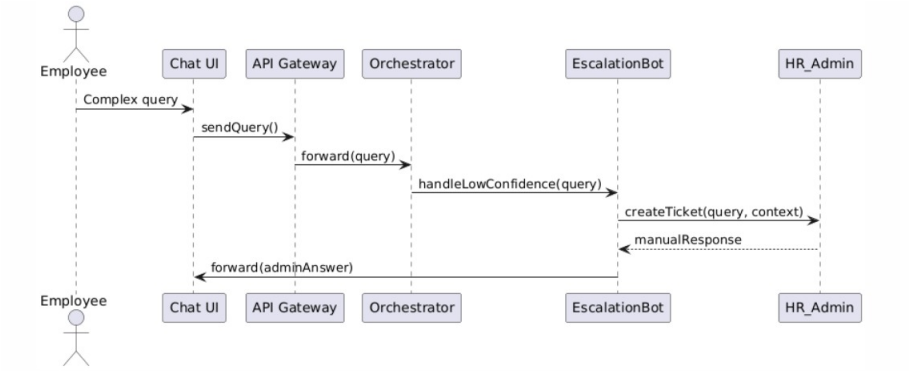


Figure 3.8: Sequence Diagram — Escalation Flow (Bot → Admin)

3.4.6 Sequence Diagram — Admin Approval Override

This sequence shows how an HR admin manually overrides an AI or manager decision.

1. HRAdmin → AdminDashboard : Select request for override
2. AdminDashboard → DatabaseModule : Fetch request details
3. DatabaseModule → AdminDashboard : Return data
4. HRAdmin → DecisionManager : Update approval
5. DecisionManager → DatabaseModule : Save changes

6. DatabaseModule → Notifier : Notify employee

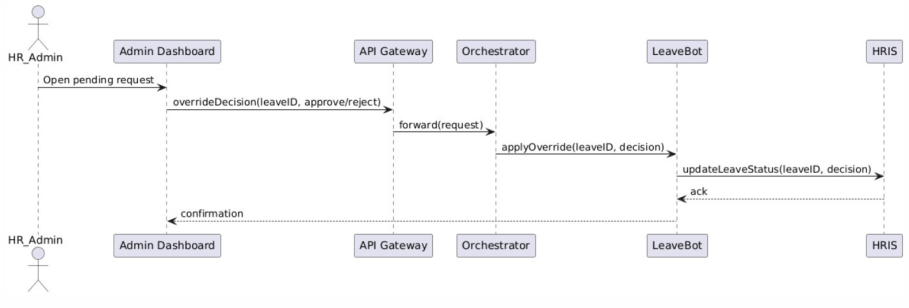


Figure 3.9: Sequence Diagram — Admin Approval Override

3.4.7 Sequence Diagram — Knowledge Base Update (Ingest → Index)

This diagram shows how the knowledge base is updated with new HR policies.

1. HRAdmin → AdminDashboard : Upload document
2. AdminDashboard → KnowledgeIngestor : Parse and validate
3. KnowledgeIngestor → TextSplitter : Chunk text
4. TextSplitter → EmbeddingModel : Vectorize content
5. EmbeddingModel → VectorDatabase : Store embeddings
6. VectorDatabase → KnowledgeIndex : Update index
7. KnowledgeIndex → HRAdmin : Confirm update

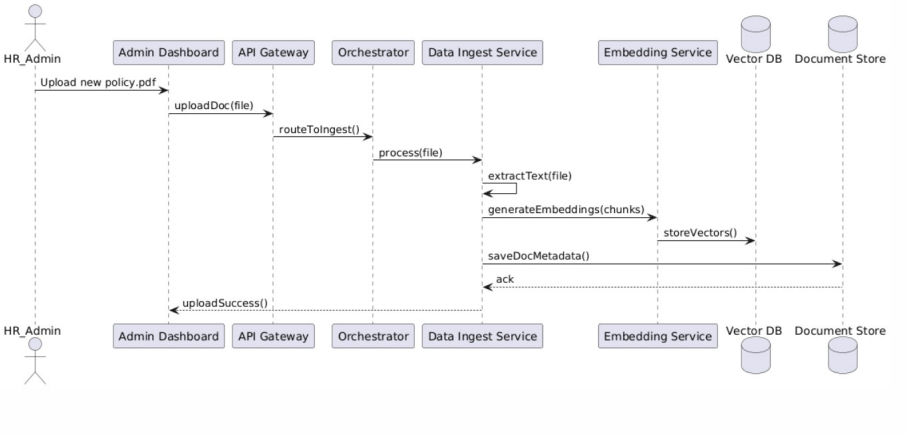


Figure 3.10: Sequence Diagram — Knowledge Base Update (Ingest → Index)

3.4.8 Sequence Diagram — Session Restore (User Reconnects)

This diagram represents how a user's previous session is restored upon reconnecting.

1. Employee → SessionHandler : Reconnect
2. SessionHandler → DatabaseModule : Retrieve session data
3. DatabaseModule → SessionHandler : Return history
4. SessionHandler → ContextManager : Load conversation
5. ContextManager → ChatInterface : Display restored chat

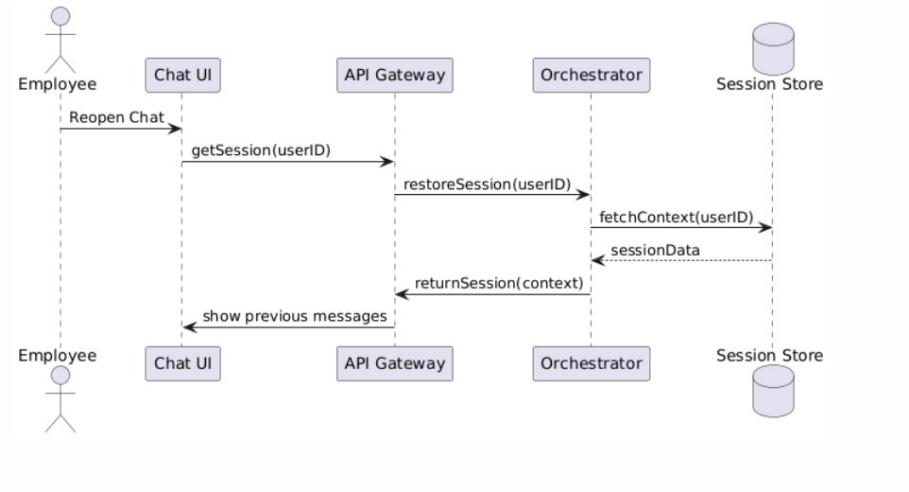


Figure 3.11: Sequence Diagram — Session Restore

3.4.9 Sequence Diagram — Analytics Export

This sequence describes how admins export analytics reports from HRFlux.

1. HRAdmin → AdminDashboard : Request analytics export
2. AdminDashboard → AnalyticsModule : Generate dataset
3. AnalyticsModule → DatabaseModule : Fetch metrics
4. DatabaseModule → AnalyticsModule : Return raw data
5. AnalyticsModule → ReportFormatter : Format report
6. ReportFormatter → FileExporter : Create export file

7. FileExporter → HRAdmin : Deliver report

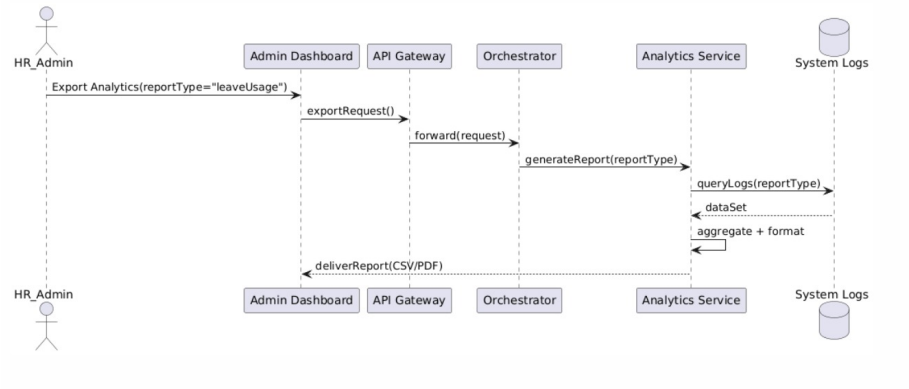


Figure 3.12: Sequence Diagram — Analytics Export

3.4.10 Data Flow Diagrams (Level 0–2)

The Data Flow Diagrams (DFDs) for HRFlux illustrate the flow of information between employees, HR managers, external HR systems, and the internal multi-agent workflow. These diagrams provide a layered understanding of how HRFlux processes user queries, manages data, and automates HR operations.

3.4.10.1 Level 0 DFD – Context Diagram

The Level 0 DFD represents the high-level overview of the HRFlux system and its interaction with external entities. It highlights the core data exchanges between employees, HR managers, and external HRIS (Human Resource Information System).

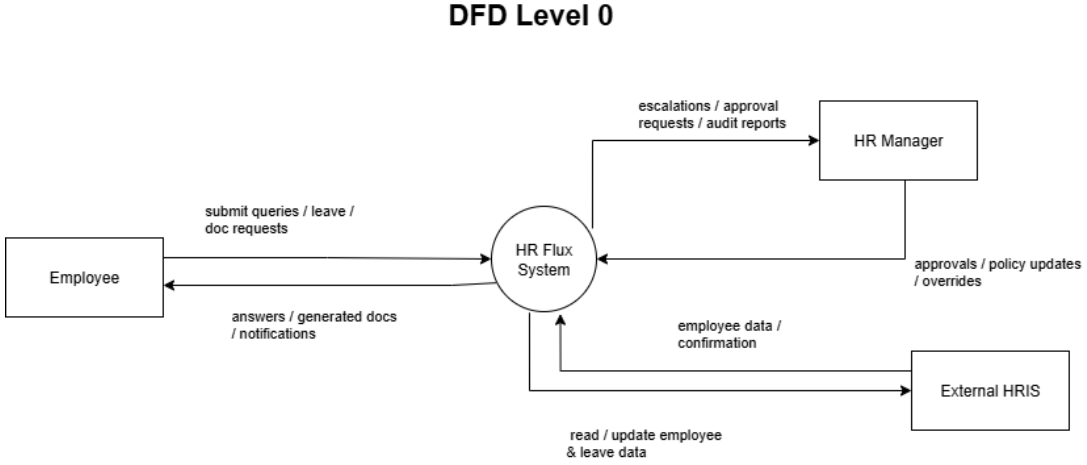


Figure 3.13: Level 0 Data Flow Diagram (Context Level) for HRFlux

Description:

- The **Employee** interacts with the HRFlux system to submit queries, leave requests, and document generation requests.
- The **HRFlux System** processes these requests and communicates with:
  - The **HR Manager** for escalations, approvals, and audit reporting.
  - The **External HRIS** for reading or updating employee data and leave information.
- The HR Manager sends back approvals, policy updates, or overrides, while the system returns answers, notifications, or generated documents to the employee.

3.4.10.2 Level 1 DFD – System Decomposition

The Level 1 DFD decomposes the HRFlux system into its main subsystems and internal data flows. It illustrates how employee queries are routed through the chatbot, processed by multiple agents, and how documents, approvals, and notifications are handled internally.

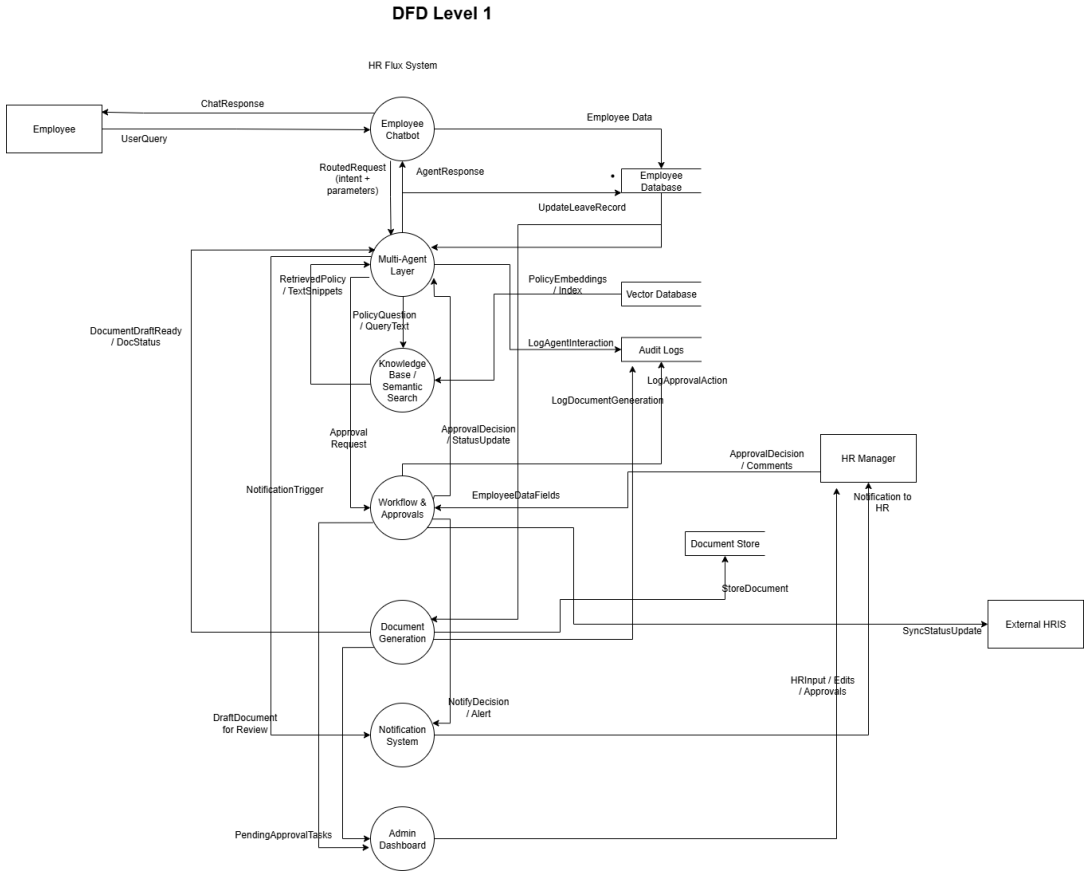


Figure 3.14: Level 1 Data Flow Diagram for HRFlux System

Processes:

1. **Employee Chatbot:** Serves as the entry point for user queries and routes requests to appropriate agents.
2. **Multi-Agent Layer:** Handles intent classification and dispatches tasks to specialized bots (LeaveBot, PolicyBot, DocuBot).
3. **Knowledge Base / Semantic Search:** Retrieves policy data and pre-indexed HR information.
4. **Workflow & Approvals:** Manages leave and document approval flows involving HR Managers and Admins.

5. **Document Generation:** Creates drafts, stores finalized documents, and triggers notifications.
6. **Notification System:** Sends alerts and status updates to employees.
7. **Admin Dashboard:** Provides oversight of pending tasks, approvals, and system logs.

Data Stores:

- **Employee Database:** Maintains employee profiles and leave records.
- **Vector Database:** Stores policy embeddings for RAG-based semantic retrieval.
- **Document Store:** Saves draft and finalized HR documents.
- **Audit Logs:** Track agent interactions, approvals, and document actions.

External Entities:

- **Employee:** Sends queries and receives automated responses.
- **HR Manager:** Handles escalations, approvals, and feedback loops.
- **External HRIS:** Synchronizes employee and leave data with HRFlux.

3.4.10.3 Level 2 DFD – Detailed Process Flows

The Level 2 DFD provides a deeper look into the internal workflows of three core subsystems: the Multi-Agent Layer, Workflow & Approvals, and Document Generator. Each process highlights detailed data movement between subprocesses and data stores.

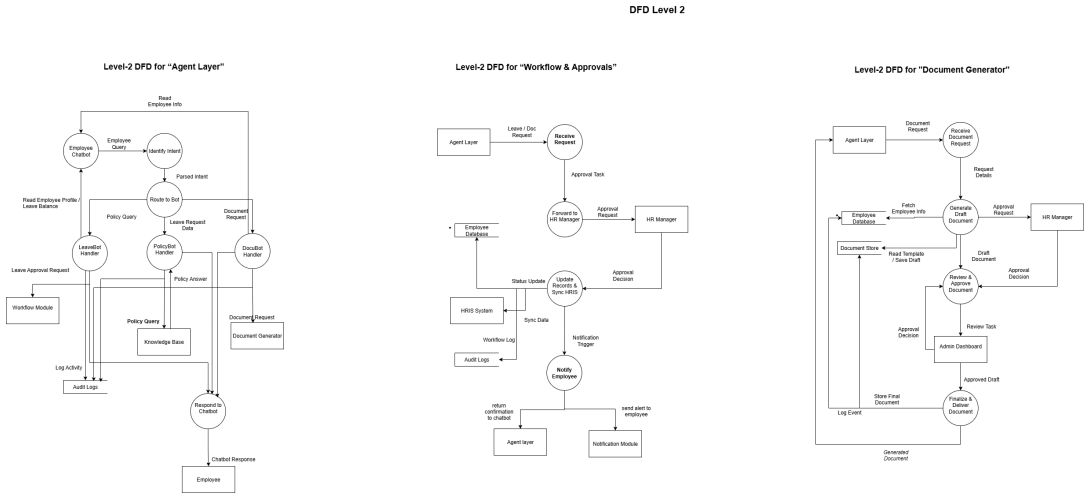


Figure 3.15: Level 2 Data Flow Diagram for HRFlux (Agent Layer, Workflow & Approvals, and Document Generator)

Detailed Descriptions:

Level-2 DFD for Agent Layer:

- The chatbot identifies user intent and routes requests to specialized bots: LeaveBot, PolicyBot, or DocuBot.
- The PolicyBot accesses the Knowledge Base to answer HR policy questions.
- The LeaveBot communicates with the Workflow Module for approval and record updates.
- The DocuBot forwards document generation requests to the Document Generator.

Level-2 DFD for Workflow & Approvals:

- Receives leave or document requests and forwards them to the HR Manager.
- The HR Manager reviews and sends back decisions (approval/rejection).
- Approved data updates the Employee Database and syncs with the External HRIS.
- Employees are notified of the final decision through the Notification Module.

Level-2 DFD for Document Generator:

- Receives document requests from the Agent Layer.
- Fetches employee information and generates a draft document using stored templates.
- HR Manager reviews and approves drafts via the Admin Dashboard.
- Approved documents are finalized, stored, and sent back to the employee as downloadable files.

Summary: The DFDs collectively depict how HRFlux automates HR operations, combining chatbot-based query handling, document automation, and HR workflow management — all synchronized with the external HRIS for consistency.

Chapter 4

Implementation and Testing

The HRFlux project has evolved into a robust Hybrid Enterprise Architecture designed to bridge unstructured knowledge management with structured HR transactions. The current implementation features a decoupled backend using FastAPI to serve REST endpoints, a dedicated Workflow Engine for business logic, and an enhanced relational database schema for employee management. The system employs a dual-pipeline approach: a Retrieval-Augmented Generation (RAG) pipeline for policy queries and a transactional state machine for leave management and approval workflows.

4.1 Algorithm Design

The core intelligence of the HRFlux chatbot resides in the `run_agent` function, which now integrates the `LeaveWorkflowEngine` to enforce business rules before database interaction. The algorithm operates on a "Traffic Controller" pattern, distinguishing between informational queries (Policies) and transactional commands (Leave Requests or Balance Checks).

If the input is classified as a leave request, the system delegates logic to the Workflow Engine. This engine performs a strict three-stage validation: checking sufficient balances, detecting scheduling overlaps, and executing atomic database writes. For general queries, the system executes a Hybrid Retrieval workflow: it merges semantic results from ChromaDB with keyword matches from the database to provide the LLM with accurate context.

```
ALGORITHM run_agent(user_id, question):
    history = get_chat_history(user_id)
    intent = classify_intent(question)

    IF intent.is_leave_request:
        extract(type, start, end, reason)

        # Validation Stage 1: Check Balances
        validity = WorkflowEngine.validate(user_id, type, start, end)
        IF NOT validity.success:
            RETURN "Error: " + validity.message

        # Validation Stage 2: Check Overlaps
        overlap = WorkflowEngine.check_overlap(user_id, start, end)
        IF overlap.exists:
            RETURN "Error: Overlapping request detected"

        # Execution: Atomic Write
        req_id = WorkflowEngine.submit(user_id, type, start, end, reason)
        RETURN "Success: Request #" + req_id + " submitted"

    ELSE IF intent.is_balance_check:
        balances = WorkflowEngine.get_balance(user_id)
        RETURN format_balance(balances)

    ELSE:
        # Hybrid Retrieval Strategy
        dense = VectorStore.search(question)
        keyword = Database.keyword_search(question)
        context = merge(dense, keyword)

        IF NOT context:
             RETURN "Information not found", suggestions

        RETURN LLM.generate(context, history, question)
```

Figure 4.1: Algorithm for the Enhanced HRFlux Agent

4.2 External APIs/SDKs

The HRFlux system leverages a robust stack of modern APIs and libraries to support its Hybrid Enterprise Architecture. In the current iteration (Modules 1 & 2), the system has transitioned to a decoupled design: the frontend utilizes Streamlit for the user interface, while the backend logic is powered by FastAPI to expose RESTful endpoints. The core intelligence integrates Google's Gemini models for intent classification and response generation, while ChromaDB handles vector storage for the RAG pipeline.

The following table summarizes the key technologies utilized in the current implementation:

API/Library	**Description**	**Purpose of usage**	**Key Function/Class used**
Streamlit	Frontend Framework	Employee Chat Interface & Admin Dashboard	`st.chat_message`, `st.session_state`
FastAPI	Backend Framework	REST API, Routing, & Workflow Management	`FastAPI`, `APIRouter`, `Depends`
Google Gemini API	Large Language Model	NLP, Intent Classification & Response Generation	`GenerativeModel.generate_content`
ChromaDB	Vector Database	Storing & Retrieving Policy Document Embeddings	`chromadb.PersistentClient`, `collection.query`
SentenceTransformers	Embedding Model	Generating Semantic Vectors for RAG	`SentenceTransformer.encode`
Pydantic	Data Validation	Request/Response Schema Validation	`BaseModel`, `Field`
SQLAlchemy	ORM Library	Database Abstraction for Enhanced Schema	`Session`, `declarative_base`

Table 4.1: Libraries and APIs used in HRFlux

4.3 Testing Details

Following the implementation of Modules 1 and 2, comprehensive testing was performed to verify the integrity of both the structured Transactional Engine and the unstructured Knowledge Base. The testing strategy focused on validating the strict architectural constraints of the workflow engine and the accuracy of the vector retrieval system.

4.3.1 Unit Testing

Unit testing was conducted on critical backend modules including `workflow_engine.py`, `db_schema_v2.py`, and `rag.py`. Each test case was designed to isolate specific logic independent of the user interface.

Key Unit Test Scenarios Covered:

- **Balance Validation:** Verified that `validate_leave_request` returns `False` when requested days (> 10) exceed the available Casual Leave balance (10).
- **Overlap Detection:** Verified that `check_overlapping_requests` correctly identifies conflicts when a new request falls within the date range of a pending application.

- **Document Ingestion (RAG):** Tested the `ingest_document` function to ensure uploaded PDF policies are correctly partitioned into 300-token chunks and stored in ChromaDB.
- **Escalation Logic:** Tested `check_and_escalate_stale_requests` to ensure requests pending for more than 7 days are correctly flagged in the `workflow_escalations` table.

4.3.2 API & Integration Testing

In addition to unit tests, the REST API endpoints defined in `backend_api.py` were tested using the Swagger UI documentation page. This ensured that the frontend Streamlit agent could successfully communicate with the backend services.

Verification Results:

- **Endpoint Reachability:** Confirmed `GET /api/employees` returns the complete list of 15 seeded employees.
- **Transaction Integrity:** Validated that a successful `POST /api/leave-requests` instantly updates the `leave_balance_history` table.

4.3.3 Functional Test Cases

The following table details the functional verification of critical workflows using the sample user account `tom.dev` (Developer profile) and password `password123`.

Test ID	Objective	Precondition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Status
TC-01	Verify API Auth	User exists in DB	Send POST request to /token	User: tom.dev Pass: password123	Return Bearer Token (200 OK)	Token generated	As Expected	**Pass**
TC-02	Verify Balance Query	User authenticated	GET request to /leave-balance	Employee ID: EMP004	JSON returns correct balances	No DB change	As Expected	**Pass**
TC-03	Verify Valid Leave	Sufficient balance available	Submit request via Chat	"Casual leave Jan 25-27"	Request ID generated	Status: Pending, Balance deducted	As Expected	**Pass**
TC-04	Verify Insufficient Balance	User has 10 casual days	Submit request $>$ 10 days	"15 days casual leave from Feb 1"	Error: "Insufficient Balance"	No Request created	As Expected	**Pass**
TC-05	Verify Overlap Detection	TC-03 is pending (Jan 25-27)	Submit overlapping dates	"Sick leave on Jan 26"	Error: "Overlap detected"	No Request created	As Expected	**Pass**
TC-06	Verify RAG Retrieval	Policies uploaded to ChromaDB	Ask policy question	"Policy for unannounced absence?"	Retrieves "Attendance Policy.pdf"	No DB change	As Expected	**Pass**

Table 4.2: Functional Test Cases for Module 1 & 2 (User: tom.dev)

4.3.3.1 Test Execution Logs

During the execution of **TC-03** (Valid Leave Submission), the backend logs confirmed the following atomic operations:

1. `LeaveWorkflowEngine`: Validated dates (2025-01-25 to 2025-01-27).
2. `Database`: Row inserted into `leave_requests_v2` with status 'Pending'.
3. `Escalation Manager`: Timer started for 7-day auto-escalation check.

Bibliography

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.14165 (GPT-3 paper).

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint*, 2018. arXiv:1810.04805.

[3] Matthijs Douze, Jeff Johnson, and Hervé Jégou. Faiss — a library for efficient similarity search. GitHub repository / technical notes, 2017.

[4] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval-augmented language model pre-training (realm). *Proceedings of the 37th International Conference on Machine Learning (PMLR)*, 2020. REALM; arXiv:2002.08909.

[5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *arXiv preprint*, 2017. arXiv:1702.08734 (FAISS-related work).

[6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[7] A Kolyshkin and S Nazarovs. Stability of slowly diverging flows in shallow water. *Mathematical Modeling and Analysis*, 2007.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive natural language processing tasks. *arXiv preprint*, 2020. arXiv:2005.11401 (RAG paper).

[9] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research / arXiv preprint (T5)*, 2020. arXiv:1910.10683.

[10] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

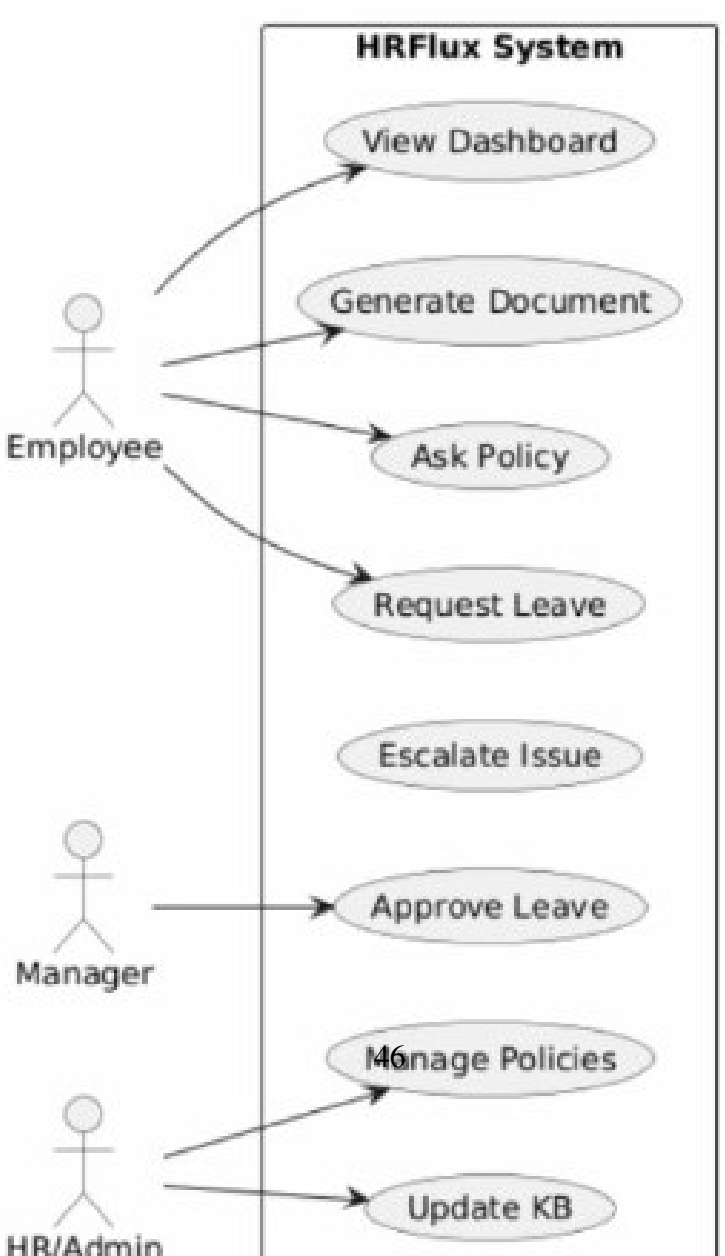
[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram)



A.2 Appendix B

A.2.1 Domain Model Example

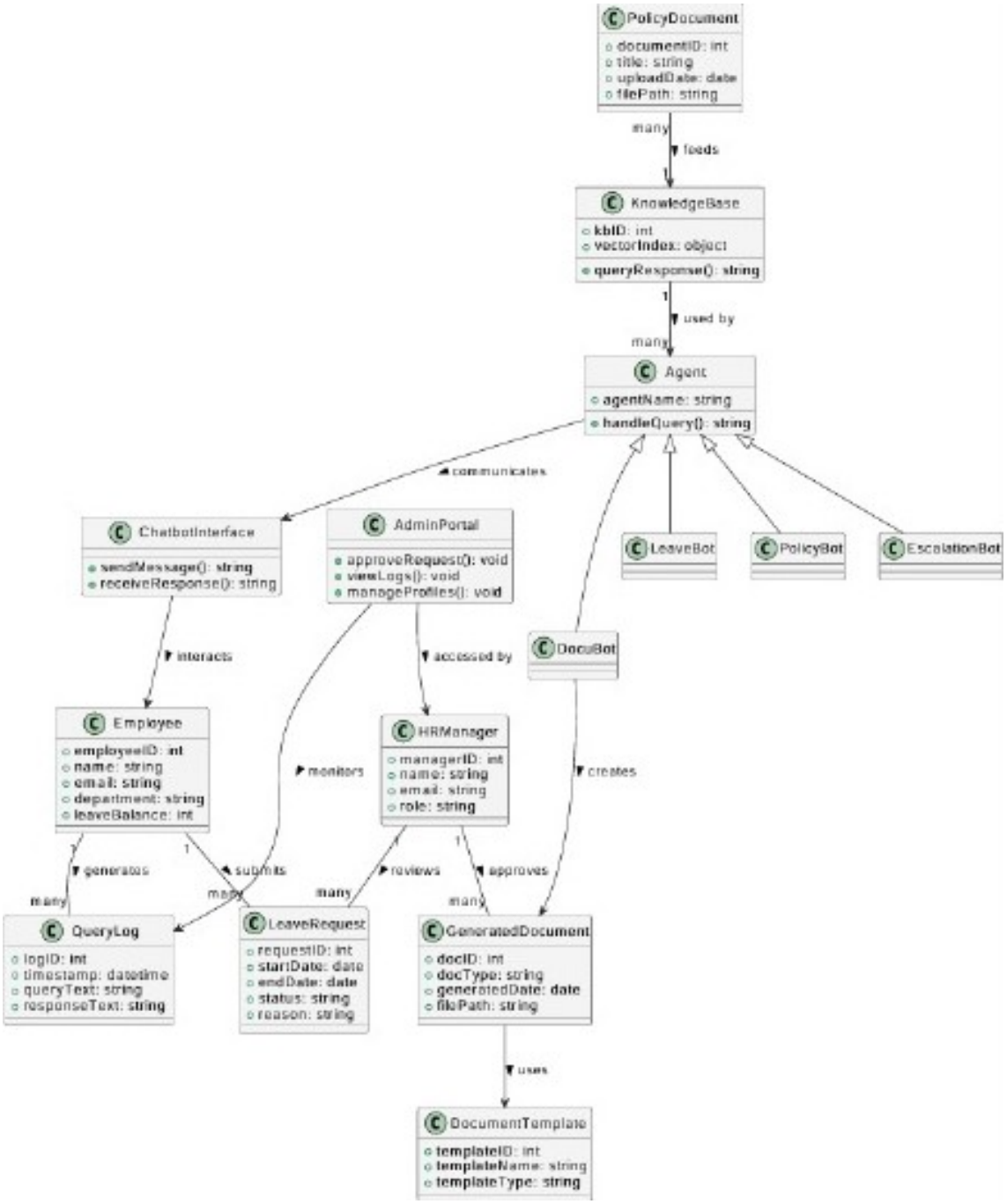


Figure A.2: Domain Model Example

A.3 Appendix C

A.3.1 Box And Line Example

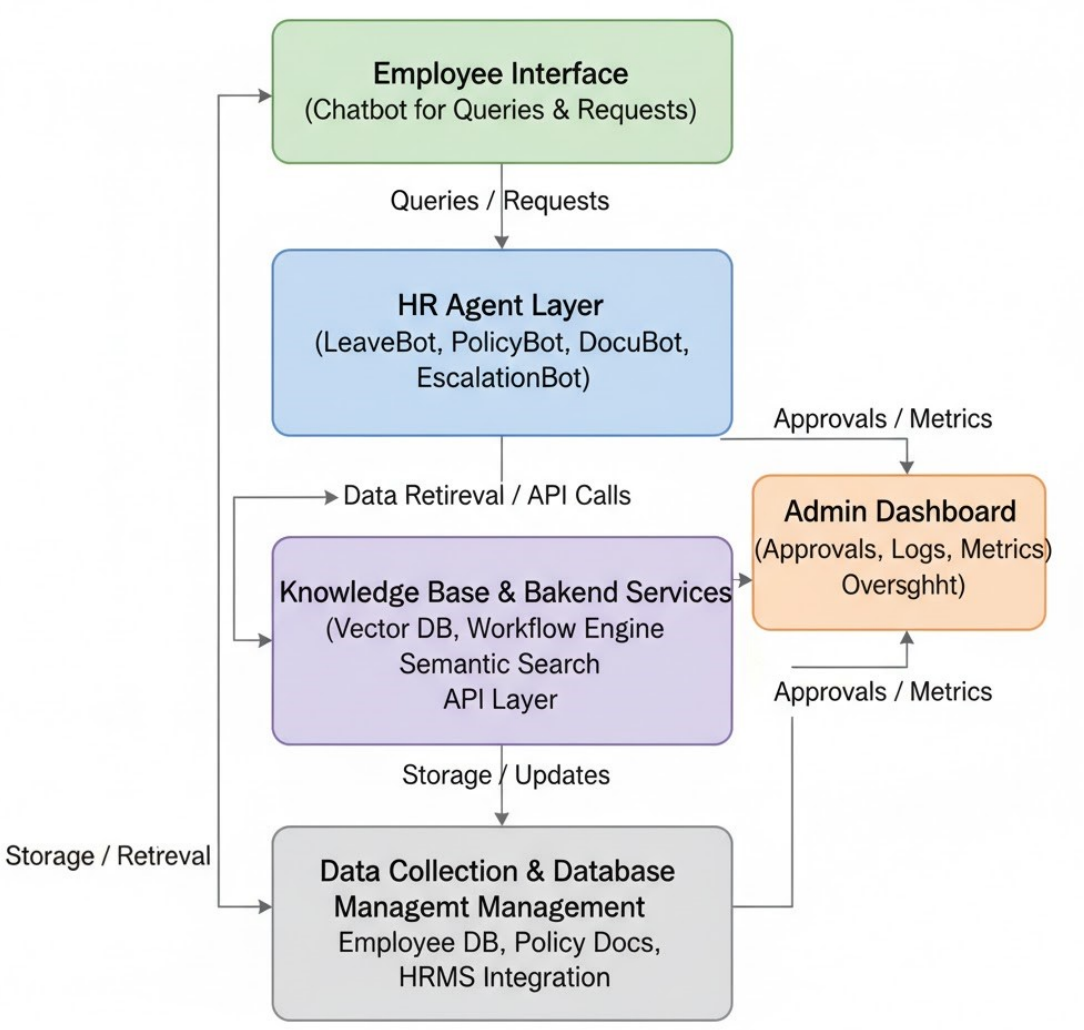


Figure A.3: Box and Line Diagram

A.3.2 Architecture Pattern

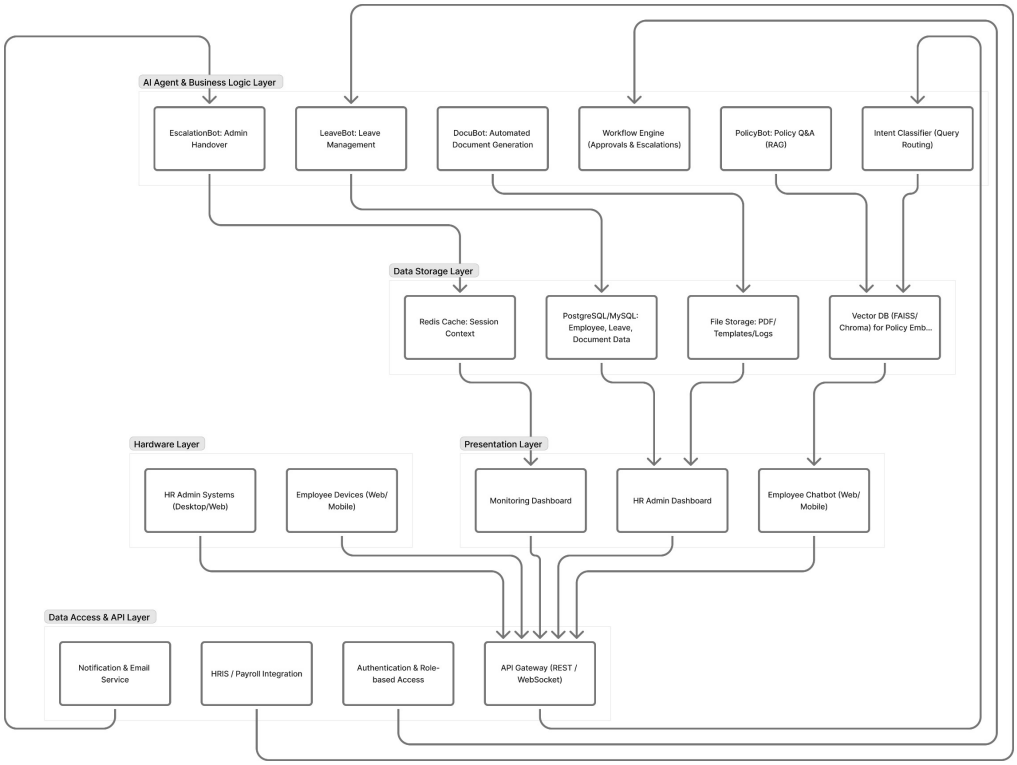


Figure A.4: Architecture Pattern

A.4 Appendix D

A.4.1 Activity Diagram

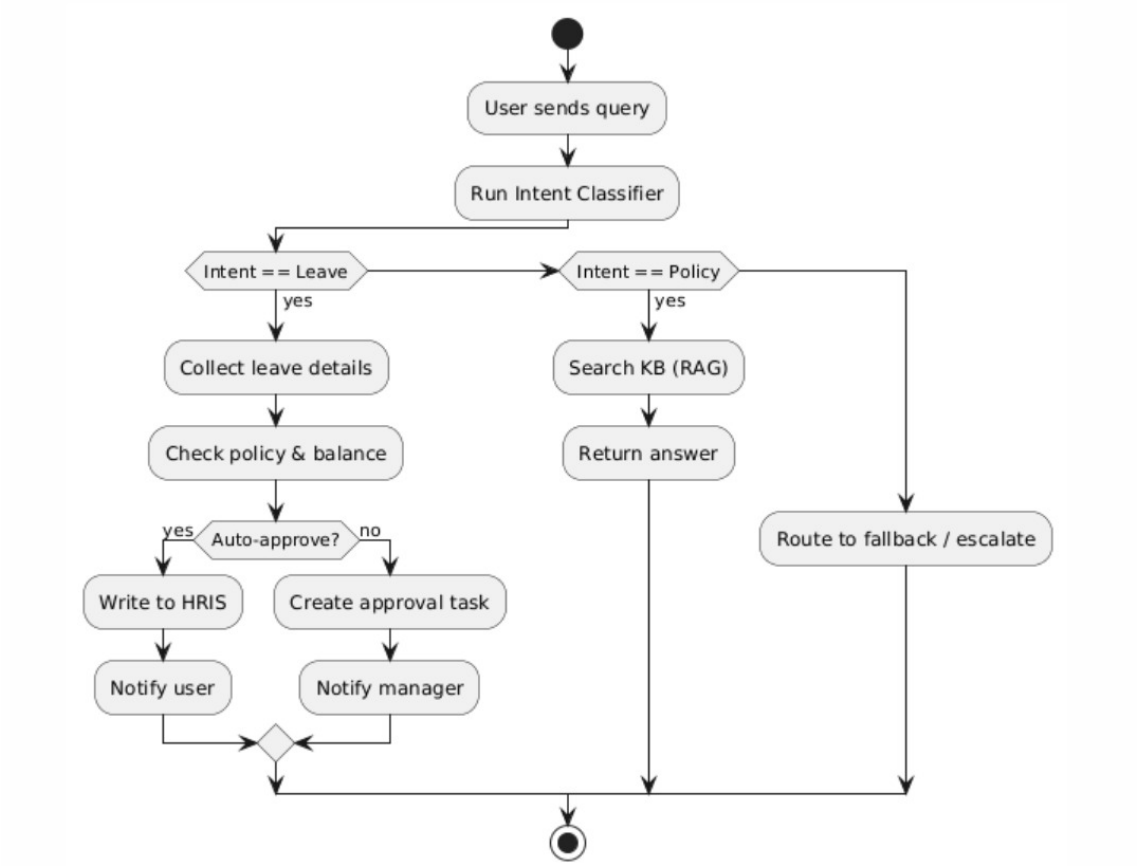


Figure A.5: Activity Diagram

A.4.2 Sequence Diagram

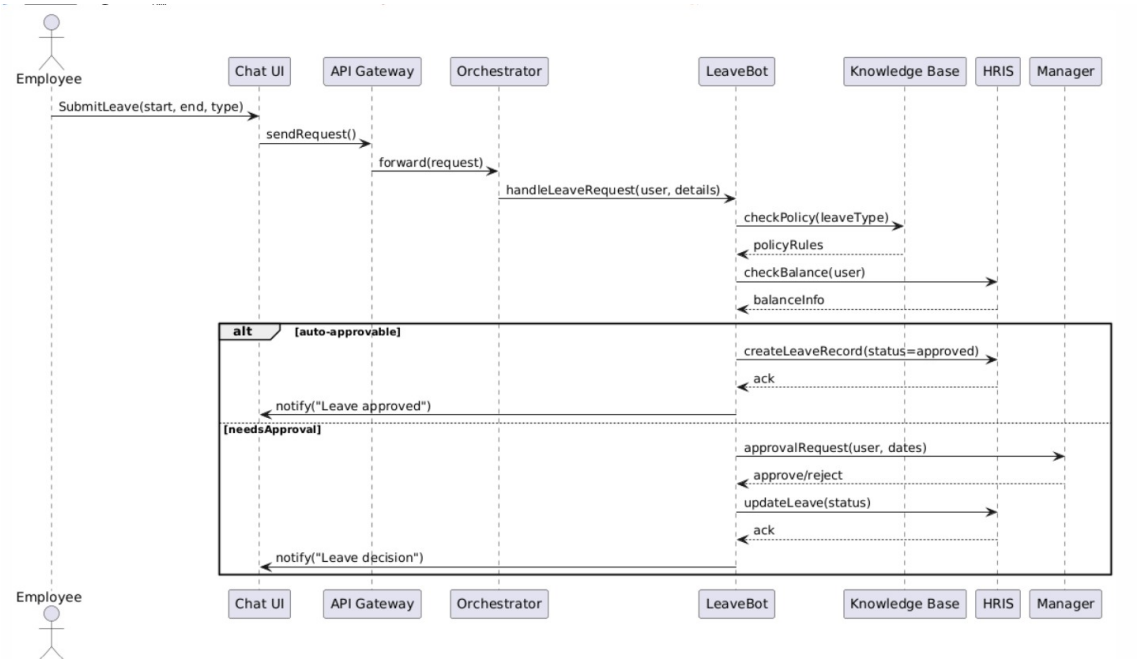


Figure A.6: Sequence Diagram For Leave Request

A.4.3 Sequence Diagram

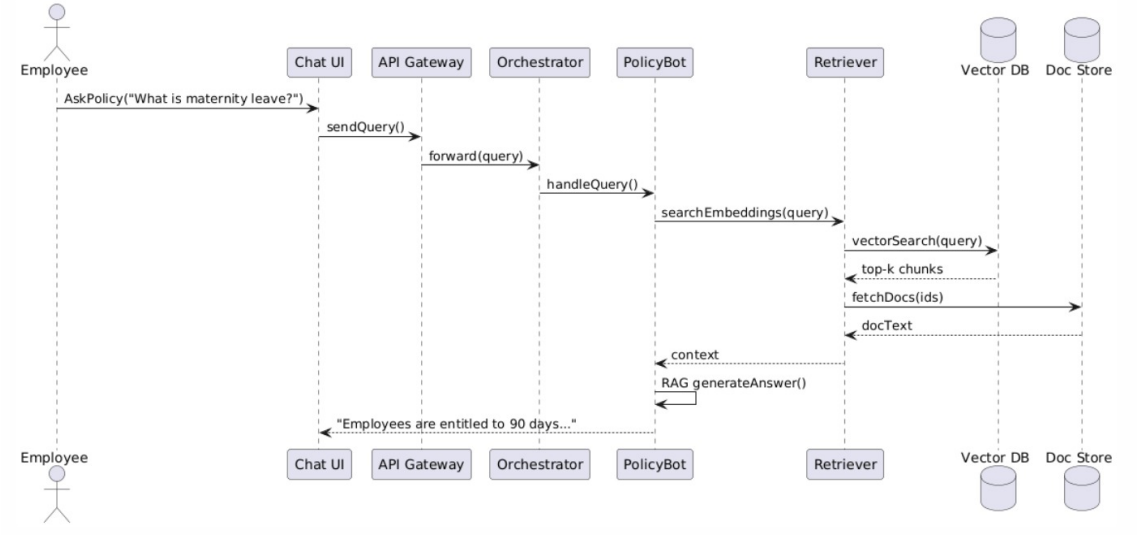


Figure A.7: Sequence Diagram For Policy Query

A.4.4 Sequence Diagram

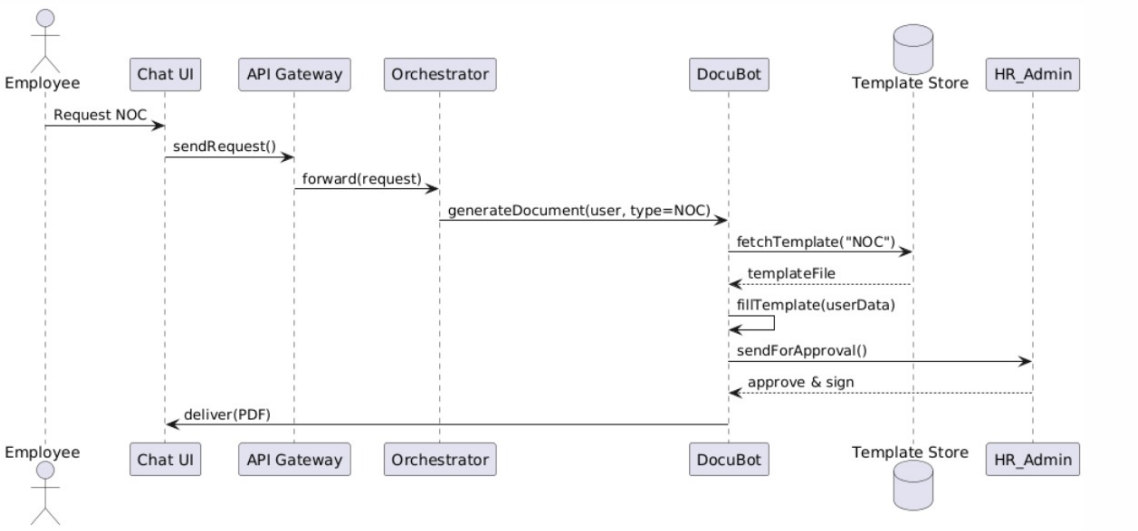


Figure A.8: Sequence Diagram For Document Generation

A.4.5 Sequence Diagram

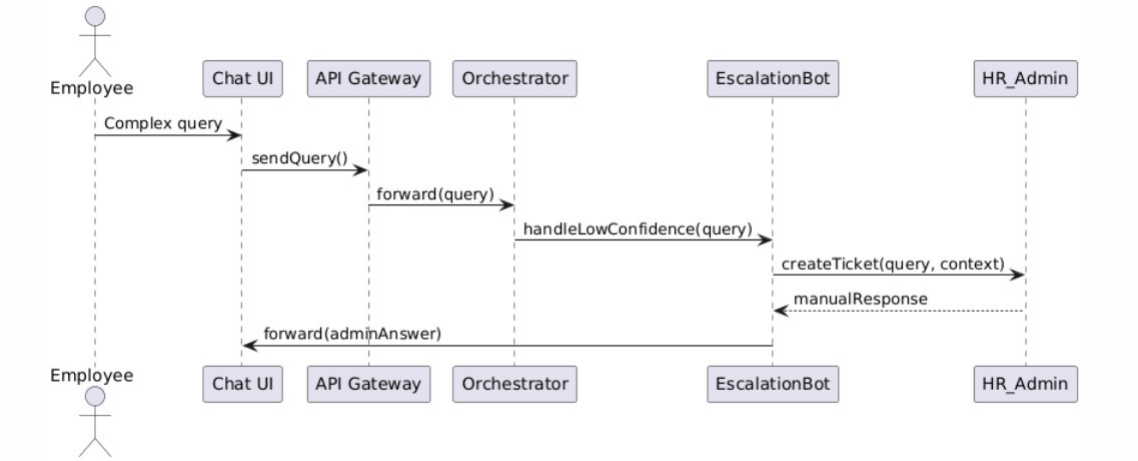


Figure A.9: Sequence Diagram For Escalation Flow

A.4.6 Sequence Diagram

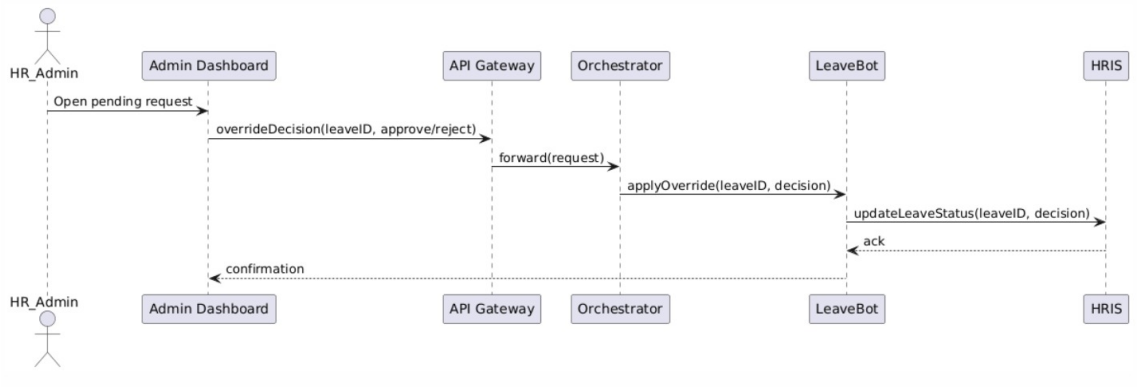


Figure A.10: Sequence Diagram For Admin Approval Override

A.4.7 Sequence Diagram

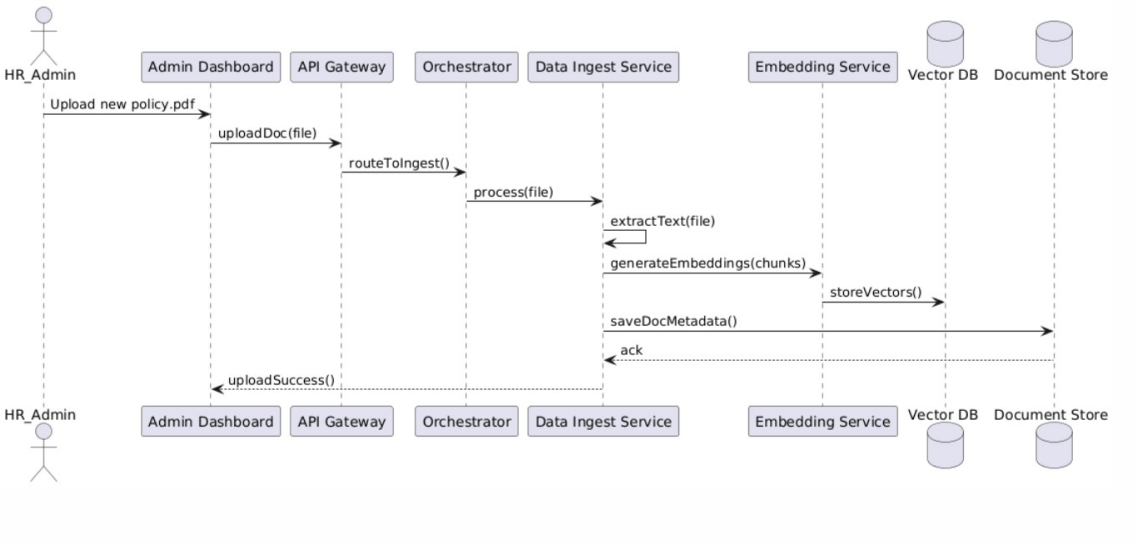


Figure A.11: Sequence Diagram For Knowledge Base Update

A.4.8 Sequence Diagram

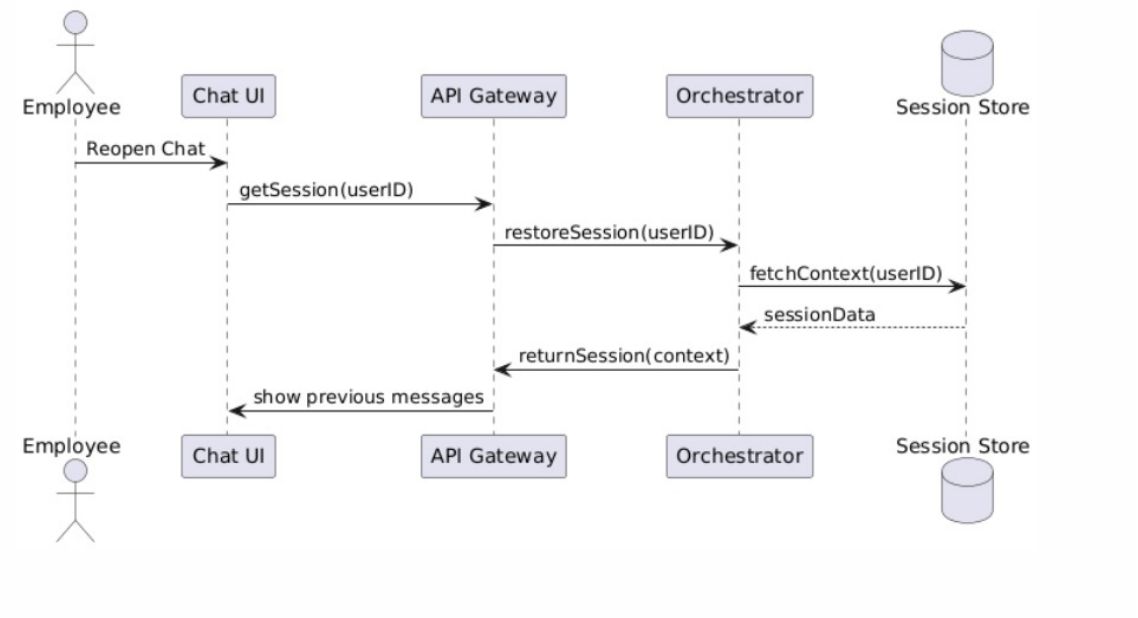


Figure A.12: Sequence Diagram For Session Restore

A.4.9 Sequence Diagram

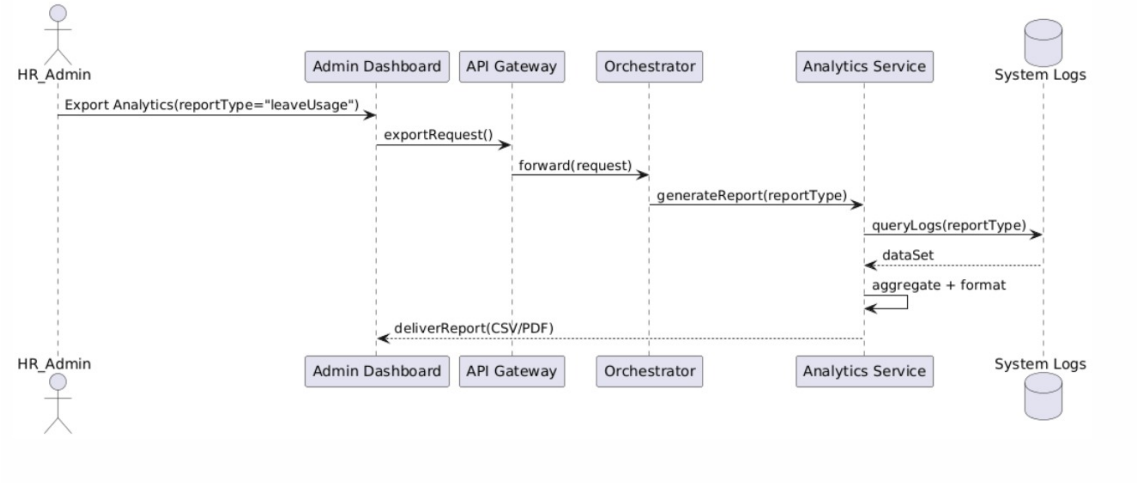
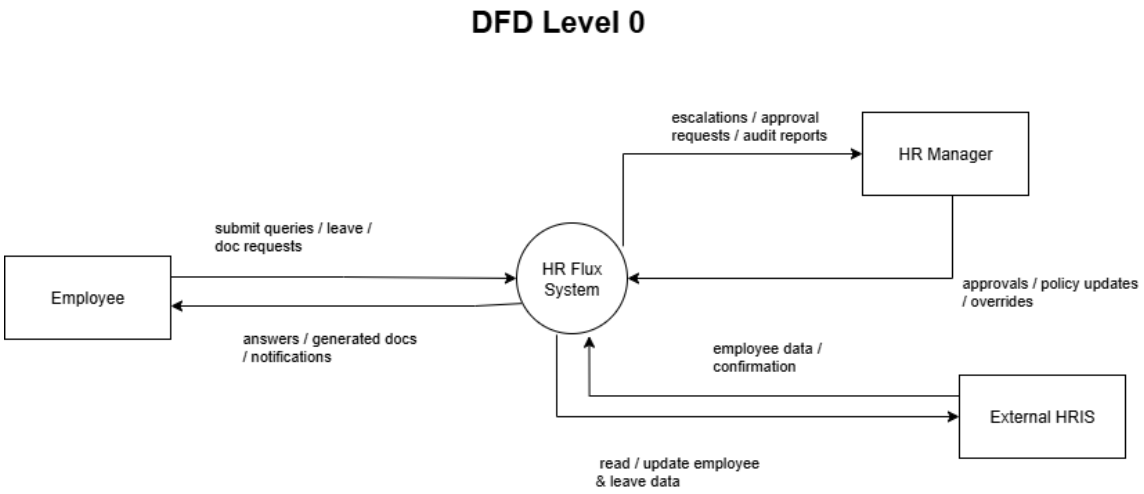
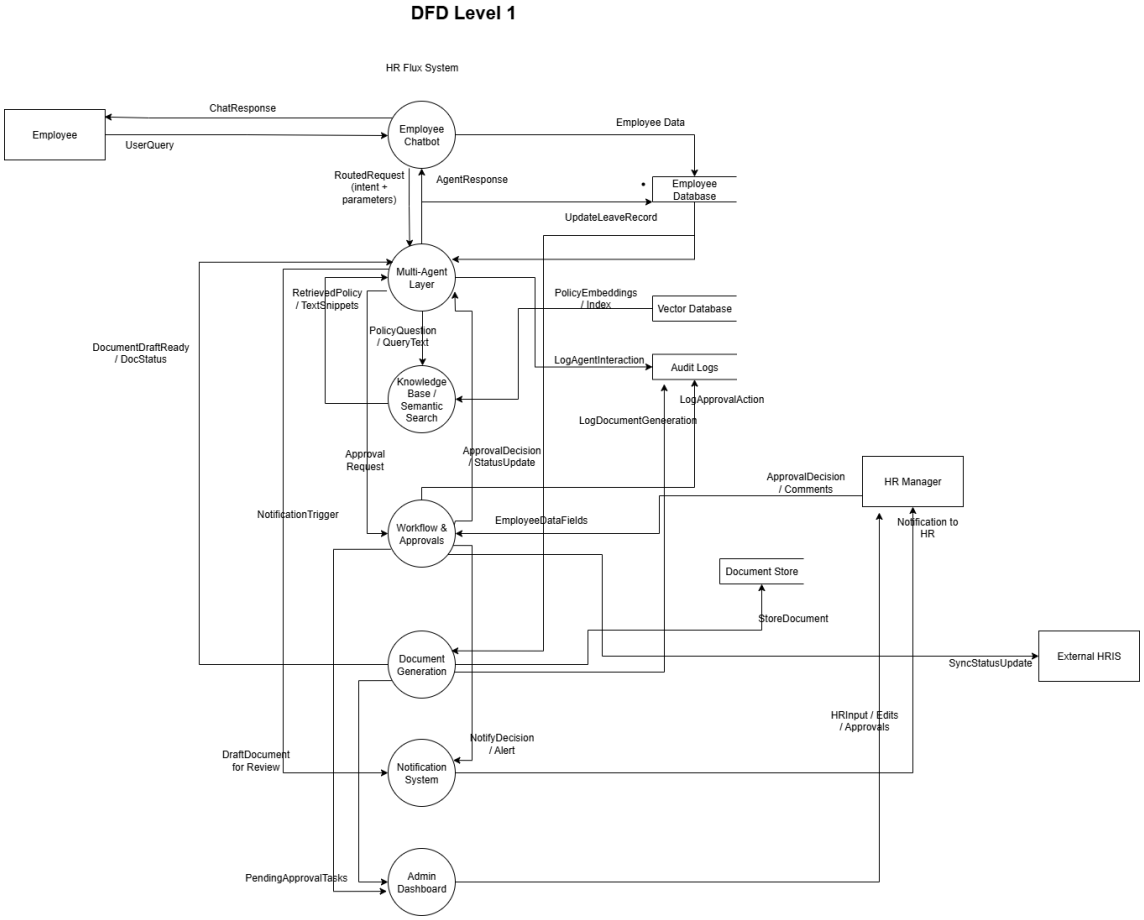


Figure A.13: Sequence Diagram For Analytics Export

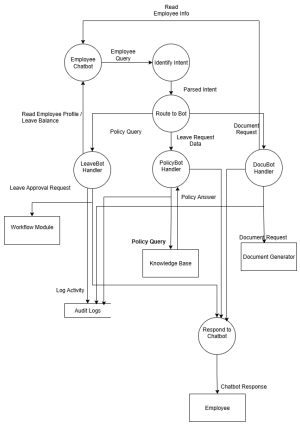
A.4.10 Data Flow Diagram



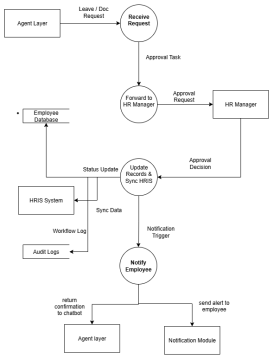


DFD Level 2

Level-2 DFD for "Agent Layer"



Level-2 DFD for "Workflow & Approvals"



Level-2 DFD for "Document Generator"